PKUCPC 模板

1 C++17 使用说明

OJ 编译标准是 C++17，使用模板中部分相关宏定义：

```cpp
#include <bits/stdc++.h>
#define endl '\n'
#define ll long long
#define ull unsigned long long
#define lowbit(x) (x & (-x))
#define rep(i, x, y) for (int i = x; i <= y; i++)
#define frep(i, x, y) for (int i = x; i >= y; i--)
#define all(x) (x).begin(), (x).end()
#define all2(x) (x).rbegin(), (x).rend()
#define sz(a) (int)a.size()
#define pii pair<int, int>
#define pll pair<ll, ll>
#define tri tuple<int, int, int>
#define trl tuple<ll, ll, ll>
#define vi vector<int>
#define vl vector<ll>
#define vvi vector<vector<int>>
#define vvl vector<vector<ll>>
#define pq priority_queue
#define umap unordered_map
#define mset multiset
using namespace std;
```

注意：模板里仍有少量 C++20/23 写法，C++17 直接交题前要替换：

- `this auto` 递归 lambda：改成 `auto dfs = [&](auto&& self, ...)` 后用 `self(self, ...)` 递归。
- `ranges::xxx`：改成普通 STL，例如 `ranges::reverse(a)` 改成 `reverse(a.begin(), a.end())`。
- `bit_width(x)`：C++17 没有。线段树可以直接开 `4 * n + 5`，树状数组二分可以手写最高位。

`this auto` 替换示例：

```cpp
// C++23 写法
auto dfs = [&](this auto&& dfs, int x, int fa) -> void {
    for (int y : g[x]) {
        if (y == fa) continue;
        dfs(y, x);
    }
};

// C++17 写法
auto dfs = [&](auto&& self, int x, int fa) -> void {
    for (int y : g[x]) {
        if (y == fa) continue;
        self(self, y, x);
    }
};
dfs(dfs, root, -1);
```

2 CPP板子

2.1 动态规划

2.1.1 数位 DP

```cpp
int count(string num1, string num2, int min_sum, int max_sum) {
    int n = num2.length();
    int dif = n - num1.length();
    const int INF = 1e9 + 7;
    map<pair<int, int>, ll> ma;
    auto dfs = [&](this auto&& dfs, int cnt, int s, bool limit_low, bool limit_high) -> int {
        if (cnt == n) return s <= max_sum && s >= min_sum;
        if (s > max_sum) return 0;
        if (!limit_low && !limit_high && ma.count({cnt, s})) return ma[{cnt, s}];
        ll res = 0;
        int lo = limit_low && cnt >= dif ? num1[cnt - dif] - '0' : 0;
        int hi = limit_high ? num2[cnt] - '0' : 9;
        int d = lo;
        if (limit_low && cnt < dif) {
            res += dfs(cnt + 1, s, true, false) % INF;
            res %= INF;
            d = 1;
        }
        for (; d <= hi; d++) {
            res += dfs(cnt + 1, s + d, limit_low && d == lo, limit_high && d == hi) % INF;
            res %= INF;
        }
        if (!limit_high && !limit_low) ma[{cnt, s}] = res;
        return res;
    };
    return dfs(0, 0, true, true);
```

```
}
// 表示给定两个整数字符串，表示上下限，以及两个整数表示数位和上下限
```

2.1.2 换根 DP

```
vector<int> lastMarkedNodes(vector<vector<int>>& edges) {
    int n = edges.size() + 1;
    vector<vector<int>> ma(n);
    for (auto& p : edges) {
        ma[p[0]].push_back(p[1]);
        ma[p[1]].push_back(p[0]);
    }
    vector<tuple<int, int, int, int>> tem(n);  // 最长，最长编号，次长，次长编号
    auto dfs = [&](this auto&& dfs, int x, int pa) -> pair<int, int> {
        int m1 = 0, m2 = 0;
        int t1 = x, t2 = x;
        for (auto& p : ma[x]) {
            if (p == pa) continue;
            auto [a, b] = dfs(p, x);
            a++;
            if (a > m1) {
                m2 = m1, t2 = t1;
                m1 = a, t1 = b;
            } else if (a > m2) {
                m2 = a;
                t2 = b;
            }
        }
        tem[x] = make_tuple(m1, t1, m2, t2);
        return {m1, t1};
    };
    dfs(0, -1);
    vector<int> res(n);    // 代表从x往上走，不经过子树的最大长度
    vector<int> up_id(n);  // 条件同上，节点编号
    vector<int> res2(n);   // 最终节点
    auto dfs2 = [&](this auto&& dfs2, int x, int pa) -> void {
        if (pa == -1) {
            res2[x] = get<1>(tem[x]);
        } else {
            int tem2 = get<0>(tem[pa]), tem3 = get<0>(tem[pa]) + 1, id = get<1>(tem[pa]);
            if (get<0>(tem[x]) + 1 == tem2 && get<1>(tem[x]) == get<1>(tem[pa])) {
                tem3 = get<2>(tem[pa]) + 1;
                id = get<3>(tem[pa]);
            }
            if (res[pa] + 1 > tem3) id = up_id[pa];
            up_id[x] = id;
            res[x] = max(tem3, res[pa] + 1);
            if (get<0>(tem[x]) > res[x]) id = get<1>(tem[x]);
            res2[x] = id;
        }
        for (int& p : ma[x]) {
            if (p == pa) continue;
            dfs2(p, x);
        }
    };
    dfs2(0, -1);
    return res2;
```

```
}
// 换根DP求每个节点为根的子树最大深度及其对应叶子
```

2.1.3 单调队列优化 DP

```cpp
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 110
#define M 100010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, W, v, w, m, h, t;
ll dp[M], dl[M][2], ans;
int main() {
    n = re(), W = re();
    for (int i = 1; i <= n; i++) {
        v = re(), w = re(), m = re();
        for (int j = 0; j < w; j++) {
            h = 1, t = 0;
            for (int k = 0; k * w + j <= W; k++) {
                ll z = dp[k * w + j] - k * v;
                while (t >= h && z > dl[t][0]) t--;
                dl[++t][0] = z;
                dl[t][1] = k;
                while (k - dl[h][1] > m) h++;
                dp[k * w + j] = max(dp[k * w + j], dl[h][0] + k * v);
            }
        }
    }
    for (int i = 1; i <= W; i++) ans = max(ans, dp[i]);
    cout << ans;
}
```

2.1.4 树形 DP 背包

```cpp
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 2010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩾ '0' && c ⩽ '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, K, x, y, he[N], tt, si[N];
ll dp[N][N], z;
struct A {
    int ne, to;
    ll z;
} b[N << 1];
void ad() {
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[tt].z = z;
    b[++tt].to = x;
    b[tt].ne = he[y];
    he[y] = tt;
    b[tt].z = z;
}
void dfs(int x, int f) {
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (v == f) continue;
        dfs(v, x);
        for (int j = min(si[x], K); j ⩾ 0; j--) {
            for (int k = min(si[v], K); k ⩾ 0; k--) {
                ll val = (ll)(k * (K - k)
                    + (si[v] - k) * (n - si[v] - K + k)) * b[i].z;
                dp[x][j + k] = max(dp[x][j + k], dp[x][j] + dp[v][k] + val);
            }
        }
        si[x] += si[v];
    }
    for (int i = min(si[x], K - 1); i ⩾ 0; i--) dp[x][i + 1] = max(dp[x][i + 1], dp[x][i]);
    si[x]++;
}
int main() {
    n = re(), K = re();
    for (int i = 1; i < n; i++) {
```

```
        x = re(), y = re(), z = re();
        ad();
    }
    dfs(1, 0);
    cout << dp[1][K];
}
```

2.1.5 动态 DP 动态树分治

```
#include <bits/stdc++.h>
#define ll long long
#define N 100100
#define M 999999999
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, m, a[N], x, y, he[N], tt, si[N], de[N], tp[N], fq[N];
int so[N], df[N], dn, dy[N], ne[2][2], ans, g[N][2], f[N][2], en[N];
struct A {
    int l, r, jz[2][2];
} tr[N << 2], o;
struct B {
    int ne, to;
} b[N << 1];
void sc(int w) {
    for (int k = 0; k < 2; k++)
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) {
                ne[i][j] = max(ne[i][j],
                    tr[w << 1 | 1].jz[i][k] + tr[w << 1].jz[k][j]);
            }
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            tr[w].jz[i][j] = ne[i][j];
            ne[i][j] = -M;
        }
}
void bu(int L, int R, int w) {
    tr[w].l = L, tr[w].r = R;
```

```
    if (L == R) {
        tr[w].jz[0][0] = tr[w].jz[1][0] = g[dy[L]][0];
        tr[w].jz[0][1] = g[dy[L]][1];
        tr[w].jz[1][1] = -M;
        return;
    }
    int mi = (L + R) >> 1;
    bu(L, mi, w << 1);
    bu(mi + 1, R, w << 1 | 1);
    sc(w);
}
A cx(int L, int R, int w) {
    if (tr[w].l >= L && tr[w].r <= R) return tr[w];
    if (tr[w << 1].r < L) return cx(L, R, w << 1 | 1);
    if (tr[w << 1 | 1].l > R) return cx(L, R, w << 1);
    A ls = cx(L, R, w << 1), rs = cx(L, R, w << 1 | 1), o;
    for (int k = 0; k < 2; k++)
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) ne[i][j] = max(ne[i][j], rs.jz[i][k] + ls.jz[k][j]);
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            o.jz[i][j] = ne[i][j];
            ne[i][j] = -M;
        }
    return o;
}
void xg(int X, int w) {
    if (tr[w].l == tr[w].r) {
        tr[w].jz[0][0] = tr[w].jz[1][0] = g[dy[X]][0];
        tr[w].jz[0][1] = g[dy[X]][1];
        tr[w].jz[1][1] = -M;
        return;
    }
    if (tr[w << 1].r >= X)
        xg(X, w << 1);
    else
        xg(X, w << 1 | 1);
    sc(w);
}
void ad() {
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[++tt].to = x;
    b[tt].ne = he[y];
    he[y] = tt;
}
void dfs(int x, int f) {
    si[x]++;
    de[x] = de[f] + 1;
    fq[x] = f;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (v == f) continue;
        dfs(v, x);
        si[x] += si[v];
        if (si[v] > si[so[x]]) so[x] = v;
    }
}
void dft(int x, int ld) {
```

```
    tp[x] = ld;
    df[x] = ++dn;
    dy[dn] = x;
    if (so[x]) {
        dft(so[x], ld);
        en[x] = en[so[x]];
    } else
        en[x] = x;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (v == fq[x] || v == so[x]) continue;
        dft(v, v);
        g[x][0] += max(f[v][0], f[v][1]);
        g[x][1] += f[v][0];
    }
    g[x][1] += a[x];
    f[x][0] = g[x][0] + max(f[so[x]][0], f[so[x]][1]);
    f[x][1] = g[x][1] + f[so[x]][0];
}
void te(int x) {
    g[x][0] = g[x][1] = 0;
    if (so[x]) te(so[x]);
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (v == fq[x] || v == so[x]) continue;
        te(v);
        g[x][0] += max(f[v][0], f[v][1]);
        g[x][1] += f[v][0];
    }
    g[x][1] += a[x];
    f[x][0] = g[x][0] + max(f[so[x]][0], f[so[x]][1]);
    f[x][1] = g[x][1] + f[so[x]][0];
}
int main() {
    n = re(), m = re();
    for (int i = 1; i <= n; i++) a[i] = re();
    for (int i = 1; i < n; i++) {
        x = re(), y = re();
        ad();
    }
    dfs(1, 0);
    dft(1, 1);
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) ne[i][j] = -M;
    bu(1, n, 1);
    for (int i = 1; i <= m; i++) {
        x = re(), y = re();
        g[x][1] -= a[x];
        a[x] = y;
        g[x][1] += a[x];
        xg(df[x], 1);
        while (x) {
            o = cx(df[tp[x]], df[en[x]], 1);
            int s1, s2, nx = fq[tp[x]];
            s1 = o.jz[0][0];
            s2 = o.jz[0][1];
            if (nx) {
                g[nx][0] += max(s1, s2) - max(f[tp[x]][0], f[tp[x]][1]);
                g[nx][1] += s1 - f[tp[x]][0];
                xg(df[nx], 1);
```

```
            } else
                ans = max(s1, s2);
            f[tp[x]][0] = s1, f[tp[x]][1] = s2;
            x = nx;
        }
        ks(ans), PC('\n');
    }
}
```

2.2 树论

2.2.1 树的直径

```
int ans = 0;
auto dfs = [&](this auto&& dfs, int x, int pa) -> int {
    int tem = 0;
    for (int& p : ma[x]) {
        if (p == pa) continue;
        int tem2 = dfs(p, x) + 1;
        // if (s[x] == s[p]) continue;
        ans = max(ans, tem + tem2);
        tem = max(tem, tem2);
    }
    return tem;
};
dfs(0, -1);
return ans; // 这里求的是边的个数，如果是点的个数需要加一
```

2.2.2 最近公共祖先 LCA

```
class TreeAncestor {
    vector<int> depth;       // 深度
    vector<vector<int>> pa;  // 2^i祖先

public:
    TreeAncestor(vector<vector<int>>& edges) {  // 直接传边数组
        int n = edges.size() + 1;
        int m = bit_width(unsigned(n));
        vector<vector<int>> ma(n);  // 这里还是0-based
        for (auto& p : edges) {
            int x = p[0], y = p[1];
            ma[x].push_back(y);
            ma[y].push_back(x);
        }
        depth.resize(n);
        pa.resize(n, vector<int>(m, -1));
        auto dfs = [&](this auto&& dfs, int x, int fa) -> void {
            pa[x][0] = fa;
            for (int y : ma[x]) {
                if (y == fa) continue;
                depth[y] = depth[x] + 1;
                dfs(y, x);
```

```
            }
            return;
        };  // 预处理深度
        dfs(0, -1);
        for (int i = 0; i < m - 1; i++) {
            for (int x = 0; x < n; x++) {
                if (pa[x][i] == -1) continue;
                pa[x][i + 1] = pa[pa[x][i]][i];
            }
        }  // 预处理2^i祖先
    }

    int get_depth(int x) { return depth[x]; }  // 获取某个点的深度

    int get_kth_ancestor(int node, int k) {
        for (int x = k; node != -1 && x > 0; x -= lowbit(x)) {
            node = pa[node][countr_zero((unsigned)x)];
        }
        return node;
    }  // 得到第k个祖先，类似树状数组的倍增

    int get_lca(int x, int y) {
        if (depth[x] > depth[y]) swap(x, y);
        y = get_kth_ancestor(y, depth[y] - depth[x]);  // 先跳到深度相同
        if (x == y) return x;
        for (int i = pa[x].size() - 1; i >= 0; i--) {
            int px = pa[x][i], py = pa[y][i];
            if (px != py) {
                x = px, y = py;
            }  // 倍增跳
        }
        return pa[x][0];
    }  // 得到最近公共祖先
};

// 简单的树上倍增模板
void solve() {
    int n;
    cin >> n;
    vi pa(n + 1);
    vvl up(n + 1, vl(20, 0));
    rep(i, 0, n) { up[i][0] = pa[i]; }
    rep(j, 0, 19) {
        rep(i, 1, n) {
            int mid = up[i][j];
            if (mid != 0) {
                up[i][j + 1] = up[mid][j];
            }
        }
    }
    return ;
}
```

2.2.3 重链剖分 HLD

思路：第一次 DFS 求父亲、深度、子树大小、重儿子；第二次 DFS 沿重链编号，得到 `dfn/top`。树上路径可以被拆成 `O(log n)` 段连续 DFS 序区间。

用法：

- `HLD hld(n); hld.add_edge(x, y); hld.work(root);`
- `hld.lca(x, y)` 查询 LCA。
- `hld.get_path(x, y)` 返回若干段 DFS 序区间，可配合线段树处理树链。
- `hld.dfn[x]..hld.dfn[x]+hld.siz[x]-1` 是 `x` 的子树区间。

```
struct HLD {
    int n, tim = 0;
    vector<vi> g;
    vi fa, dep, siz, son, top, dfn, rnk;

    HLD(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n, {});
        fa.assign(n, -1);
        dep.assign(n, 0);
        siz.assign(n, 0);
        son.assign(n, -1);
        top.assign(n, 0);
        dfn.assign(n, 0);
        rnk.assign(n, 0);
        tim = 0;
    }

    void add_edge(int x, int y) {
        g[x].push_back(y);
        g[y].push_back(x);
    }

    void work(int root = 0) {
        auto dfs1 = [&](auto&& self, int x, int p) -> void {
            fa[x] = p;
            siz[x] = 1;
            for (int y : g[x]) {
                if (y == p) continue;
                dep[y] = dep[x] + 1;
                self(self, y, x);
                siz[x] += siz[y];
                if (son[x] == -1 || siz[y] > siz[son[x]]) son[x] = y;
            }
        };
        auto dfs2 = [&](auto&& self, int x, int tp) -> void {
            top[x] = tp;
            dfn[x] = tim;
            rnk[tim++] = x;
            if (son[x] != -1) self(self, son[x], tp);
            for (int y : g[x]) {
                if (y == fa[x] || y == son[x]) continue;
                self(self, y, y);
            }
        };
```

```
        dfs1(dfs1, root, -1);
        dfs2(dfs2, root, root);
    }

    int lca(int x, int y) {
        while (top[x] != top[y]) {
            if (dep[top[x]] < dep[top[y]]) swap(x, y);
            x = fa[top[x]];
        }
        return dep[x] < dep[y] ? x : y;
    }

    int dist(int x, int y) {
        int z = lca(x, y);
        return dep[x] + dep[y] - 2 * dep[z];
    }

    vector<pii> get_path(int x, int y) {
        vector<pii> res;
        while (top[x] != top[y]) {
            if (dep[top[x]] < dep[top[y]]) swap(x, y);
            res.push_back({dfn[top[x]], dfn[x]});
            x = fa[top[x]];
        }
        if (dep[x] > dep[y]) swap(x, y);
        res.push_back({dfn[x], dfn[y]});
        return res;
    }
};
```

2.2.4 虚树

思路：把关键点按 DFS 序排序，相邻关键点的 LCA 一定覆盖所有需要补的点。
加入这些 LCA 后，用栈按祖先关系连边，得到只包含关键点和必要 LCA 的压缩树。

用法：

- 需要先有 HLD 的 `dfn/siz/dep/lca`。
- `VirtualTree vt(n, hld); auto nodes = vt.build(key);`
- `vt.g` 是虚树有向边，边从祖先指向儿子，边权是原树距离。
- 每次 `build` 会清空上次用到的虚树边。

```
vt.build(key)  // 建虚树，返回虚树点集
vt.g[u]        // u 的虚树儿子，pair 是 {儿子, 原树距离}
nodes[0]       // 虚树根
vt.used        // 当前虚树点集，主要内部清空用
struct VirtualTree {
    int n;
    HLD& hld;
    vector<vector<pii>> g;
```

```
    vi used;

    VirtualTree(int n, HLD& hld) : n(n), hld(hld), g(n) {}

    bool is_ancestor(int x, int y) {
        return hld.dfn[x] <= hld.dfn[y] && hld.dfn[y] < hld.dfn[x] + hld.siz[x];
    }

    vi build(vi key) {
        for (int x : used) g[x].clear();
        used.clear();
        if (key.empty()) return {};

        sort(all(key), [&](int x, int y) { return hld.dfn[x] < hld.dfn[y]; });
        key.erase(unique(all(key)), key.end());
        int m = sz(key);
        rep(i, 0, m - 2) key.push_back(hld.lca(key[i], key[i + 1]));
        sort(all(key), [&](int x, int y) { return hld.dfn[x] < hld.dfn[y]; });
        key.erase(unique(all(key)), key.end());

        vi st;
        for (int x : key) {
            while (!st.empty() && !is_ancestor(st.back(), x)) st.pop_back();
            if (!st.empty()) {
                g[st.back()].push_back({x, hld.dep[x] - hld.dep[st.back()]});
            }
            st.push_back(x);
        }
        used = key;
        return used;
    }
};
```

2.2.5 点分治

思路：每次找当前连通块的重心，处理所有经过重心的路径贡献，再递归处理删掉重心后的子树。每层子问题大小至少减半，所以复杂度常见是 `O(n log n)`。

用法：

- `CentroidDecomposition cd(n); cd.add_edge(x, y); cd.work();`
- `cd.par[x]` 是点分树父亲，根的父亲是 `-1`。
- 真正做题时，把“经过重心的统计”写在 `build` 里标注的位置。

```
struct CentroidDecomposition {
    int n;
    vector<vi> g;
    vi siz, par, vis;

    CentroidDecomposition(int n = 0) { init(n); }
```

```
    void init(int n_) {
        n = n_;
        g.assign(n, {});
        siz.assign(n, 0);
        par.assign(n, -1);
        vis.assign(n, 0);
    }

    void add_edge(int x, int y) {
        g[x].push_back(y);
        g[y].push_back(x);
    }

    int get_size(int x, int p) {
        siz[x] = 1;
        for (int y : g[x]) {
            if (y == p || vis[y]) continue;
            siz[x] += get_size(y, x);
        }
        return siz[x];
    }

    int get_centroid(int x, int p, int tot) {
        for (int y : g[x]) {
            if (y == p || vis[y]) continue;
            if (siz[y] * 2 > tot) return get_centroid(y, x, tot);
        }
        return x;
    }

    void build(int x, int p) {
        int tot = get_size(x, -1);
        int c = get_centroid(x, -1, tot);
        par[c] = p;
        vis[c] = 1;

        // 在这里处理经过重心 c 的路径贡献

        for (int y : g[c]) {
            if (vis[y]) continue;
            build(y, c);
        }
    }

    void work(int root = 0) {
        build(root, -1);
    }
};
```

2.2.6 点分树

```
#include <bits/stdc++.h>
#define ll long long
#define N 100100
#define M 99999999
#define GC getchar()
#define PC putchar
```

```
using namespace std;
int re() {
    int s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩽ '9' && c ⩾ '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(int s) {
    if (s < 0) PC('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, q, he[N], tt, ci, x, y, a[N], zz[N][18], de[N], si[N], mx[N], pa;
int zx, vi[N], fa[N];
struct A {
    int ne, to;
} b[N << 1];
struct B {
    int ls, rs, z;
} ai;
struct C {
    int ro[N], gs;
    B d[N * 50];
    void xg(int w, int L, int R, int X, int z) {
        if (L == R) {
            d[w].z += z;
            return;
        }
        int mi = (L + R) >> 1;
        if (X ⩽ mi) {
            if (!d[w].ls) d[w].ls = ++gs;
            xg(d[w].ls, L, mi, X, z);
        } else {
            if (!d[w].rs) d[w].rs = ++gs;
            xg(d[w].rs, mi + 1, R, X, z);
        }
        d[w].z = d[d[w].ls].z + d[d[w].rs].z;
    }
    int cx(int w, int L, int R, int X) {
        if (!w) return 0;
        if (R ⩽ X) return d[w].z;
        int s = 0, mi = (L + R) >> 1;
        s += cx(d[w].ls, L, mi, X);
        if (mi < X) s += cx(d[w].rs, mi + 1, R, X);
        return s;
    }
} tr, ts;
void ad() {
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[++tt].to = x;
    b[tt].ne = he[y];
```

```
    he[y] = tt;
}
void pdf(int x, int f) {
    zz[x][0] = f;
    de[x] = de[f] + 1;
    for (int i = 1; i < 18; i++) zz[x][i] = zz[zz[x][i - 1]][i - 1];
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (v == f) continue;
        pdf(v, x);
    }
}
void dfs(int x, int f, int ds) {
    si[x] = 1;
    mx[x] = 0;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (vi[v] || v == f) continue;
        dfs(v, x, ds);
        mx[x] = max(mx[x], si[v]);
        si[x] += si[v];
    }
    mx[x] = max(mx[x], ds - si[x]);
    if (mx[x] < mx[zx]) zx = x;
}
void dft(int x, int ds) {
    vi[x] = 1;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (vi[v]) continue;
        zx = 0;
        int siz = (si[v] < si[x]) ? si[v] : ds - si[x];
        dfs(v, x, siz);
        fa[zx] = x;
        dft(zx, siz);
    }
}
int lca(int x, int y) {
    if (de[x] < de[y]) swap(x, y);
    int s = 0, c = de[x] - de[y];
    while (c) {
        if (c & 1) x = zz[x][s];
        s++;
        c >>= 1;
    }
    if (x == y) return x;
    for (int i = 17; i >= 0; i--)
        if (zz[x][i] ^ zz[y][i]) x = zz[x][i], y = zz[y][i];
    return zz[x][0];
}
int g_d(int x, int y) { return de[x] + de[y] - (de[lca(x, y)] << 1); }
void dxg(int x, int z) {
    int cu = x;
    while (cu) {
        tr.xg(tr.ro[cu], 0, n - 1, g_d(cu, x), z);
        if (fa[cu]) ts.xg(ts.ro[cu], 0, n - 1, g_d(fa[cu], x), z);
        cu = fa[cu];
    }
}
int dcx(int x, int k) {
```

```
    int cu = x, pr = 0, s = 0;
    while (cu) {
        if (g_d(cu, x) > k) {
            pr = cu;
            cu = fa[cu];
            continue;
        }
        s += tr.cx(tr.ro[cu], 0, n - 1, k - g_d(cu, x));
        if (pr) s -= ts.cx(ts.ro[pr], 0, n - 1, k - g_d(cu, x));
        pr = cu;
        cu = fa[cu];
    }
    return s;
}
int main() {
    n = re();
    q = re();
    for (int i = 1; i <= n; i++) a[i] = re();
    for (int i = 1; i < n; i++) {
        x = re(), y = re();
        ad();
    }
    pdf(1, 0);
    mx[0] = M;
    dfs(1, 0, n);
    dft(zx, n);
    for (int i = 1; i <= n; i++) tr.ro[i] = ts.ro[i] = i;
    tr.gs = n, ts.gs = n;
    for (int i = 1; i <= n; i++) dxg(i, a[i]);
    while (q--) {
        ci = re(), x = re(), y = re();
        x ^= pa;
        y ^= pa;
        if (ci) {
            dxg(x, y - a[x]);
            a[x] = y;
        } else {
            pa = dcx(x, y);
            ks(pa);
            PC('\n');
        }
    }
}
```

2.3 数据结构

2.3.1 分块

```
void solve() {
    int n;
    cin >> n;
    int B = sqrt(n);
    vector<ll> a(n);
    vector<ll> sum((n + B - 1) / B);
    vector<ll> tag((n + B - 1) / B);
```

```cpp
rep(i, 0, n - 1) cin >> a[i];
auto build = [&]() { rep(i, 0, n - 1) sum[i / B] += a[i]; };

// 单点加法
auto add = [&](int pos, int val) {
    a[pos] += val;
    sum[pos / B] += val;
};

// 区间加法
auto add = [&](int l, int r, int val) {
    int idl = l / B, idr = r / B;
    if (idl == idr) {
        rep(i, l, r) {
            a[i] += val;
            sum[idl] += val;
        }
    } else {
        rep(i, l, (idl + 1) * B - 1) {
            a[i] += val;
            sum[idl] += val;
        }
        rep(i, idl + 1, idr - 1) {
            tag[i] += val;
            sum[i] += 1LL * val * B;
        }
        rep(i, idr * B, r) {
            a[i] += val;
            sum[idr] += val;
        }
    }
};

// 区间查询
auto query = [&](int l, int r) -> ll {
    ll res = 0;
    int idl = l / B, idr = r / B;
    if (idl == idr) {
        rep(i, l, r) res += a[i];
    } else {
        rep(i, l, (idl + 1) * B - 1) res += a[i];
        rep(i, idl + 1, idr - 1) res += sum[i];
        rep(i, idr * B, r) res += a[i];
    }
    return res;
};

// 带标记区间查询
auto query = [&](int l, int r) {
    ll res = 0;
    int idl = l / B, idr = r / B;
    if (idl == idr) {
        rep(i, l, r) res += a[i] + tag[idl];
    } else {
        rep(i, l, (idl + 1) * B - 1) res += a[i] + tag[idl];
        rep(i, idl + 1, idr - 1) res += sum[i];
        rep(i, idr * B, r) res += a[i] + tag[idr];
    }
    return res;
```

```
    };
}
```

2.3.2 莫队

思路：把询问按左端点所在块排序，块内按右端点蛇形排序。维护当前区间 `[L, R]`，每次移动端点并增删贡献。

用法：

- 下面模板求每个询问区间内不同数个数。
- `a` 是 0-indexed，`qs` 里的 `[l, r]` 也是 0-indexed 闭区间。
- 如果要换题，只改 `add/del` 里面维护的贡献。

```
struct MoQuery {
    int l, r, id;
};

vi mo_distinct(vi a, vector<pii> qs) {
    int n = sz(a), q = sz(qs);
    vi b = a;
    sort(all(b));
    b.erase(unique(all(b)), b.end());
    for (int& x : a) x = lower_bound(all(b), x) - b.begin();

    int B = max(1, (int)sqrt(n));
    vector<MoQuery> query(q);
    rep(i, 0, q - 1) query[i] = {qs[i].first, qs[i].second, i};
    sort(all(query), [&](const MoQuery& x, const MoQuery& y) {
        int bx = x.l / B, by = y.l / B;
        if (bx != by) return bx < by;
        return bx & 1 ? x.r > y.r : x.r < y.r;
    });

    vi cnt(sz(b)), ans(q);
    int L = 0, R = -1, cur = 0;
    auto add = [&](int p) {
        if (++cnt[a[p]] == 1) cur++;
    };
    auto del = [&](int p) {
        if (--cnt[a[p]] == 0) cur--;
    };

    for (auto [l, r, id] : query) {
        while (L > l) add(--L);
        while (R < r) add(++R);
        while (L < l) del(L++);
        while (R > r) del(R--);
        ans[id] = cur;
    }
```

```
    return ans;
}
```

2.3.3 回滚莫队

思路：普通莫队需要支持 `add/del`。如果删除不好做，就按左端点分块。
询问右端点只向右扩展，根据块右端点分段，左端点临时暴力加入，回答后回滚到快照。

用法：

- 下面模板求每个询问区间内元素最高出现次数。
- 适合“加入容易，删除困难，但状态能回滚”的题。
- `a` 是 0-indexed，`qs` 里的 `[l, r]` 是 0-indexed 闭区间。

```
struct RollbackQuery {
    int l, r, id;
};

vi rollback_mo_max_freq(vi a, vector<pii> qs) {
    int n = sz(a), q = sz(qs);
    vi b = a;
    sort(all(b));
    b.erase(unique(all(b)), b.end());
    for (int& x : a) x = lower_bound(all(b), x) - b.begin();

    int B = max(1, (int)sqrt(n));
    vector<vector<RollbackQuery>> bucket((n + B - 1) / B);
    rep(i, 0, q - 1) {
        auto [l, r] = qs[i];
        bucket[l / B].push_back({l, r, i});
    }

    vi ans(q), cnt(sz(b));
    rep(id, 0, sz(bucket) - 1) {
        auto& v = bucket[id];
        sort(all(v), [&](const RollbackQuery& x, const RollbackQuery& y) { return x.r < y.r; });

        int br = min(n - 1, (id + 1) * B - 1);
        int R = br, cur = 0;
        fill(all(cnt), 0);

        auto add = [&](int p) {
            cnt[a[p]]++;
            cur = max(cur, cnt[a[p]]);
        };

        for (auto [l, r, qid] : v) {
            if (r <= br) {
                int res = 0;
                rep(i, l, r) {
                    cnt[a[i]]++;
```

```
                res = max(res, cnt[a[i]]);
            }
            rep(i, l, r) cnt[a[i]]--;
            ans[qid] = res;
            continue;
        }

        while (R < r) add(++R);
        int old = cur;
        rep(i, l, br) add(i);
        ans[qid] = cur;
        rep(i, l, br) cnt[a[i]]--;
        cur = old;
    }
  }
  return ans;
}
```

2.3.4 笛卡尔树（我的）

```
// 笛卡尔树，默认为大根堆
template <class T, typename Compare = std::greater<T>>
struct CartesianTree {
    T inf = std::numeric_limits<T>::max();

    struct Node {
        int idx;                // 原数组下标
        T val;                  // 权值
        int par, siz;           // 父节点索引，子树大小
        std::array<int, 2> son;  // 左儿子，右儿子

        Node(int idx = 0, T val = 0, int par = 0, int siz = 0)
            : idx(idx), val(val), par(par), siz(siz), son{} {}
    };

    std::vector<Node> t;

    CartesianTree() { init(); }

    void init(Compare comp = Compare()) {
        t.assign(1, {0, 0, 0});
        t[0].son.fill(0);
        if (comp(-inf, inf))
            t[0].val = -inf;
        else
            t[0].val = inf;
    }  // 自动建立虚拟节点

    // 负责把元素加进末尾，需要使用1-based
    void add(int idx, T val, int par = 0) { t.emplace_back(idx, val, par); }

    int work(Compare comp = Compare()) {
        for (int i = 1; i < t.size(); i++) {
            int k = i - 1;
            while (comp(t[i].val, t[k].val)) k = t[k].par;
            t[i].son[0] = t[k].son[1];
            t[k].son[1] = i;
```

```
            t[i].par = k;
            t[t[i].son[0]].par = i;
        }  // 遍历，砍树枝
        auto dfs = [&](auto &&dfs, int u) -> void {
            if (!u) return;
            t[u].siz = 1;
            dfs(dfs, ls(u));
            dfs(dfs, rs(u));
            t[u].siz += t[ls(u)].siz + t[rs(u)].siz;
        };  // 进行一个dfs
        dfs(dfs, t[0].son[1]);
        return t[0].son[1];
    }

    int Left(int p) { return p - size(ls(p)); }  // 左边最远

    int Right(int p) { return p + size(rs(p)); }  // 右边最远

    int size(int p) { return t[p].siz; }

    int ls(int p) { return t[p].son[0]; }

    int rs(int p) { return t[p].son[1]; }

    int par(int p) { return t[p].par; }
};

// 使用示例
CartesianTree<int, less<int>> ct;

rep(i, 0, n - 1) { ct.add(i + 1, nums[i]); }

int root = ct.work(less<int>());
rep(i, 1, n) {
    int L = ct.Left(i);
    int R = ct.Right(i);
}
```

2.3.5 主席树

```
using ll = long long;

struct ChairmanTree {
    struct Node {
        int ls = 0, rs = 0, cnt = 0;
        ll sum = 0;
    };

    vector<Node> tr;
    vector<int> root;
    vector<int> vals;

    ChairmanTree(const vector<int>& a) {
        vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
```

```
        int n = a.size();
        tr.reserve((n + 1) * 20);
        tr.push_back(Node());
        root.assign(n + 1, 0);
        root[0] = build(1, vals.size());
        for (int i = 1; i <= n; i++) {
            int pos = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin() + 1;
            root[i] = add(root[i - 1], 1, vals.size(), pos, a[i - 1]);
        }
    }

    int build(int l, int r) {
        int o = new_node();
        if (l == r) return o;
        int m = (l + r) >> 1;
        tr[o].ls = build(l, m);
        tr[o].rs = build(m + 1, r);
        return o;
    }

    int add(int old, int l, int r, int p, int val) {
        int o = copy_node(old);
        tr[o].cnt++;
        tr[o].sum += val;
        if (l == r) return o;
        int m = (l + r) >> 1;
        if (p <= m) tr[o].ls = add(tr[old].ls, l, m, p, val);
        else tr[o].rs = add(tr[old].rs, m + 1, r, p, val);
        return o;
    }

    int kth(int l, int r, int k) const {
        return vals[kth(root[l - 1], root[r], 1, vals.size(), k) - 1];
    }

    pair<int, ll> leq(int l, int r, int x) const {
        int pos = upper_bound(vals.begin(), vals.end(), x) - vals.begin();
        if (!pos) return {0, 0};
        return query(root[l - 1], root[r], 1, vals.size(), pos);
    }

private:
    int new_node() {
        tr.push_back(Node());
        return tr.size() - 1;
    }

    int copy_node(int old) {
        tr.push_back(tr[old]);
        return tr.size() - 1;
    }

    int kth(int old, int now, int l, int r, int k) const {
        if (l == r) return l;
        int left_cnt = tr[tr[now].ls].cnt - tr[tr[old].ls].cnt;
        int m = (l + r) >> 1;
        if (k <= left_cnt) return kth(tr[old].ls, tr[now].ls, l, m, k);
        return kth(tr[old].rs, tr[now].rs, m + 1, r, k - left_cnt);
    }
```

```
    pair<int, ll> query(int old, int now, int l, int r, int qr) const {
        if (r <= qr) return {tr[now].cnt - tr[old].cnt, tr[now].sum - tr[old].sum};
        int m = (l + r) >> 1;
        auto res = query(tr[old].ls, tr[now].ls, l, m, qr);
        if (qr > m) {
            auto t = query(tr[old].rs, tr[now].rs, m + 1, r, qr);
            res.first += t.first;
            res.second += t.second;
        }
        return res;
    }
};

// 使用：ChairmanTree ct(a);
// ct.kth(l, r, k)：查询 1-indexed 区间 [l, r] 第 k 小的原值
// ct.leq(l, r, x)：查询区间 [l, r] 内 <= x 的数量和总和
//
// vector<int> a(n);              // a 是 0-indexed 原数组
// ChairmanTree ct(a);
// int kth_val = ct.kth(l, r, k); // l, r, k 都按 1-indexed 传
// auto [cnt, sum] = ct.leq(l, r, limit);
```

2.3.6 懒标记线段树

```
template <typename T, typename F>
class LazySegmentTree {
    const F TODO_INIT = 0;  // 懒标记初始值
    struct Node {
        T val;
        F todo;
    };
    int n;
    vector<Node> tree;
    T merge_val(const T& a, const T& b) const { return a + b; }   // 合并两个val
    F merge_todo(const F& a, const F& b) const { return a + b; }  // 合并两个懒标记
    void apply(int node, int l, int r, F todo) {
        Node& cur = tree[node];
        cur.val += todo * (r - l + 1);
        cur.todo = merge_todo(todo, cur.todo);
    }  // 把懒标记作用到node子树
    void pushdown(int node, int l, int r) {
        Node& cur = tree[node];
        F todo = cur.todo;
        if (todo == TODO_INIT) return;
        int m = (l + r) >> 1;
        apply(node << 1, l, m, todo);
        apply(node << 1 | 1, m + 1, r, todo);
        cur.todo = TODO_INIT;
    }  // 把当前节点的懒标记下传
    void maintain(int node) { tree[node].val = merge_val(tree[node << 1].val, tree[node << 1 | 1].val); }
    // 合并线段树
    void build(const vector<T>& a, int node, int l, int r) {
        Node& cur = tree[node];
        cur.todo = TODO_INIT;
        if (l == r) {
            cur.val = a[l];
            return;
```

```
        }
        int m = (l + r) >> 1;
        build(a, node << 1, l, m);
        build(a, node << 1 | 1, m + 1, r);
        maintain(node);
    }  // 建树，复杂度O(n)
    void update(int node, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) {
            apply(node, l, r, f);
            return;
        }
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) update(node << 1, l, m, ql, qr, f);
        if (qr > m) update(node << 1 | 1, m + 1, r, ql, qr, f);
        maintain(node);
    }  // 区间更新[ql,qr]
    T query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[node].val;
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        if (qr <= m) return query(node << 1, l, m, ql, qr);
        if (ql > m) return query(node << 1 | 1, m + 1, r, ql, qr);
        T l_res = query(node << 1, l, m, ql, qr);
        T r_res = query(node << 1 | 1, m + 1, r, ql, qr);
        return merge_val(l_res, r_res);
    }  // 区间查找
    int find_first(int node, int l, int r, int ql, int qr, T val) {
        if (r < ql || l > qr) return -1;
        if (tree[node].val < val) return -1;
        if (l == r) return l;
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        int res = find_first(node << 1, l, m, ql, qr, val);
        if (res != -1) return res;
        return find_first(node << 1 | 1, m + 1, r, ql, qr, val);
    }
    // 若固定左端点，需要记录前缀分段最大值，并加被待求区间完全覆盖的剪枝

    int find_last(int node, int l, int r, int ql, int qr, T val) {
        if (r < ql || l > qr) return -1;
        if (tree[node].val < val) return -1;
        if (l == r) return l;
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        int res = find_last(node << 1 | 1, m + 1, r, ql, qr, val);
        if (res != -1) return res;
        return find_last(node << 1, l, m, ql, qr, val);
    }

public:
    LazySegmentTree(int n, T init_val = 0) : LazySegmentTree(vector<T>(n, init_val)) {}
    // 维护下标为[0,n-1],初始值为init_val的区间，或者数组a
    LazySegmentTree(const vector<T>& a)
        : n(a.size()), tree(2 << bit_width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }
    // 更新[ql,qr]为f
    void update(int ql, int qr, F f) { update(1, 0, n - 1, ql, qr, f); }
    // 区间查询[ql,qr]
```

```
    T query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }

    // 查询[ql,qr]中第一个满足条件的下标
    int find_first(int ql, int qr, T val) { return find_first(1, 0, n - 1, ql, qr, val); }

    // 查询[ql,qr]中最后一个满足条件的下标
    int find_last(int ql, int qr, T val) { return find_last(1, 0, n - 1, ql, qr, val); }
};
// 注：懒标记线段树无论做什么都需要pushdown
// 此时其它与线段树二分同

// 双标记，注意顺序
const int MOD = 1e9 + 7;
template <typename T, typename F>
class LazySegmentTree {
    const F TODO_INIT = 0;  // 懒标记初始值
    const F TODO_INIT2 = 1;
    struct Node {
        T val;
        F todo;
        F todo2;
    };
    int n;
    vector<Node> tree;
    T merge_val(const T& a, const T& b) const { return (a + b) % MOD; }  // 合并两个val
    F merge_todo(const F& a, const F& b, const F& c) const {
        return (a * c % MOD + b % MOD) % MOD;  // 合并两个懒标记
    }
    F merge_todo2(const F& a, const F& b) const {
        return (a * b) % MOD;  // 合并两个乘法标记
    }
    void apply(int node, int l, int r, F todo, F todo2) {
        Node& cur = tree[node];
        cur.val *= todo2;
        cur.val %= MOD;
        cur.val += (todo % MOD) * ((r - l + 1) % MOD) % MOD;
        cur.val %= MOD;
        cur.todo = merge_todo(cur.todo, todo, todo2);
        cur.todo2 = merge_todo2(todo2, cur.todo2);
    }  // 把懒标记作用到node子树
    void pushdown(int node, int l, int r) {
        Node& cur = tree[node];
        F todo = cur.todo;
        F todo2 = cur.todo2;
        if (todo == TODO_INIT && todo2 == TODO_INIT2) return;
        int m = (l + r) >> 1;
        apply(node << 1, l, m, todo, todo2);
        apply(node << 1 | 1, m + 1, r, todo, todo2);
        cur.todo = TODO_INIT;
        cur.todo2 = TODO_INIT2;
    }  // 把当前节点的懒标记下传
    void maintain(int node) { tree[node].val = merge_val(tree[node << 1].val, tree[node << 1 | 1].val); }
    // 合并线段树
    void build(const vector<T>& a, int node, int l, int r) {
        Node& cur = tree[node];
        cur.todo = TODO_INIT;
        cur.todo2 = TODO_INIT2;
        if (l == r) {
            cur.val = a[l];
            return;
```

```
        }
        int m = (l + r) >> 1;
        build(a, node << 1, l, m);
        build(a, node << 1 | 1, m + 1, r);
        maintain(node);
    }  // 建树，复杂度O(n)
    void update(int node, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) {
            apply(node, l, r, f, 1);
            return;
        }
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) update(node << 1, l, m, ql, qr, f);
        if (qr > m) update(node << 1 | 1, m + 1, r, ql, qr, f);
        maintain(node);
    }  // 区间更新[ql,qr]
    void update2(int node, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) {
            apply(node, l, r, 0, f);
            return;
        }
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) update2(node << 1, l, m, ql, qr, f);
        if (qr > m) update2(node << 1 | 1, m + 1, r, ql, qr, f);
        maintain(node);
    }
    T query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[node].val;
        pushdown(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) return query(node << 1, l, m, ql, qr);
        if (ql > m) return query(node << 1 | 1, m + 1, r, ql, qr);
        T l_res = query(node << 1, l, m, ql, qr) % MOD;
        T r_res = query(node << 1 | 1, m + 1, r, ql, qr) % MOD;
        return merge_val(l_res, r_res);
    }  // 区间查找

public:
    LazySegmentTree(int n, T init_val = 0) : LazySegmentTree(vector<T>(n, init_val)) {}
    // 维护下标为[0,n-1],初始值为init_val的区间，或者数组a
    LazySegmentTree(const vector<T>& a)
        : n(a.size()), tree(2 << bit_width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }
    // 更新[ql,qr]为f
    void update(int ql, int qr, F f) { update(1, 0, n - 1, ql, qr, f); }
    // 更新[ql,qr]为f
    void update2(int ql, int qr, F f) { update2(1, 0, n - 1, ql, qr, f); }
    // 区间查询[ql,qr]
    T query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }
};

// LC850:矩形面积并（扫描线）
const ll MOD = 1e9 + 7;
class SegmentTree {
private:
    struct Node {
        int l, r;
```

```
        int min_cover_len = 0;  // 区间内被覆盖的最小次数
        int min_cover = 0;      // 区间内为最小次数的区间长度
        int todo = 0;           // 懒标记
    };

    vector<Node> seg;

    void maintain(int o) {
        Node& lo = seg[o << 1];
        Node& ro = seg[(o << 1) | 1];
        int mn = min(lo.min_cover, ro.min_cover);
        seg[o].min_cover = mn;
        seg[o].min_cover_len =
            (lo.min_cover == mn ? lo.min_cover_len : 0)
            + (ro.min_cover == mn ? ro.min_cover_len : 0);
    }  // 根据左右儿子的信息，更新当前节点的信息

    void do_(int o, int v) {
        seg[o].min_cover += v;
        seg[o].todo += v;
    }  // 仅更新节点信息，不下传懒标记

    void pushdown(int o) {
        int& v = seg[o].todo;
        if (v) {
            do_(o << 1, v);
            do_(o << 1 | 1, v);
            v = 0;
        }
    }  // 下传懒标记

    void build(vi& xs, int o, int l, int r) {
        seg[o].l = l;
        seg[o].r = r;
        if (l == r) {
            seg[o].min_cover_len = xs[l + 1] - xs[l];
            return;
        }
        int m = (l + r) >> 1;
        build(xs, o << 1, l, m);
        build(xs, o << 1 | 1, m + 1, r);
        maintain(o);
        return;
    }

    void update(int o, int l, int r, int v) {
        if (l <= seg[o].l && seg[o].r <= r) {
            do_(o, v);
            return;
        }
        pushdown(o);
        int m = (seg[o].l + seg[o].r) >> 1;
        if (l <= m) update(o << 1, l, r, v);
        if (m < r) update(o << 1 | 1, l, r, v);
        maintain(o);
    }

public:
    SegmentTree(vi& xs) {
        unsigned n = sz(xs) - 1;  // 有这么多个差值
```

```
        seg.resize(2 << bit_width(n - 1));
        build(xs, 1, 0, n - 1);  // 根节点是1
    }

    void update(int l, int r, int v) { update(1, l, r, v); }

    int get_uncovered_length() { return seg[1].min_cover ? 0 : seg[1].min_cover_len; }
};
class Solution {
public:
    int rectangleArea(vector<vector<int>>& rectangles) {
        int n = sz(rectangles);
        vi xs;
        struct Event {
            int y, lx, rx, d;
        };
        vector<Event> tem;
        for (auto& p : rectangles) {
            int lx = p[0], rx = p[2], ly = p[1], ry = p[3];
            xs.push_back(lx);
            xs.push_back(rx);
            tem.emplace_back(ly, lx, rx, 1);
            tem.emplace_back(ry, lx, rx, -1);
        }
        ranges::sort(xs);
        xs.erase(unique(all(xs)), xs.end());
        SegmentTree tree(xs);
        sort(all(tem), [&](const Event& x, const Event& y) { return x.y < y.y; });
        int m = sz(tem);
        ll ans = 0;
        rep(i, 0, m - 2) {
            auto& [y, lx, rx, d] = tem[i];
            int l = ranges::lower_bound(xs, lx) - xs.begin();
            int r = ranges::lower_bound(xs, rx) - xs.begin() - 1;  // 注意点和区间的对应关系
            tree.update(l, r, d);
            int sum = xs.back() - xs[0] - tree.get_uncovered_length();
            ans += 1LL * sum * (tem[i + 1].y - y);
            ans %= MOD;
        }
        return ans % MOD;
    }
};
```

2.3.7 单调栈

```
int largestRectangleArea(vector<int>& nums) {
    int n = nums.size();
    vector<int> r(n, n);
    vector<int> l(n, -1);
    stack<int> s;
    for (int i = n - 1; i ⩾ 0; i--) {
        while (!s.empty() && nums[s.top()] ⩾ nums[i]) s.pop();
        if (!s.empty()) r[i] = s.top();
        s.push(i);
    } // 求右边第一个小于的下标
    while (!s.empty()) s.pop();
    for (int i = 0; i ⩽ n - 1; i++) {
```

```
        while (!s.empty() && nums[s.top()] >= nums[i]) s.pop();
        if (!s.empty()) l[i] = s.top();
        s.push(i);
    } // 求左边第一个小于的下标
    int maxx = INT_MIN;
    for (int i = 0; i <= n - 1; i++) maxx = max(maxx, nums[i] * (r[i] - l[i] - 1));
    return maxx;
}
```

2.3.8 线段树

```
struct Info {
    ll sum, pre, suf, ans;
    Info(ll x = 0) : sum(x), pre(x), suf(x), ans(x) {}
};
Info operator+(const Info& a, const Info& b) {
    Info c;
    c.sum = a.sum + b.sum;
    c.pre = max(a.pre, a.sum + b.pre);
    c.suf = max(b.suf, b.sum + a.suf);
    c.ans = max({a.ans, b.ans, a.suf + b.pre});
    return c;
}
template <typename T>
class SegmentTree {
    int n;
    vector<T> tree;
    T merge_val(T a, T b) const { return a + b; }  // 合并子树

    void maintain(int node) {  // 维护整棵树
        tree[node] = merge_val(tree[node * 2], tree[node * 2 + 1]);
    }

    void build(const vector<T>& a, int node, int l, int r) {
        if (l == r) {
            tree[node] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        maintain(node);
    }  // 建树

    void update(int node, int l, int r, int i, T val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int m = (l + r) / 2;
        if (i <= m)
            update(node * 2, l, m, i, val);
        else
            update(node * 2 + 1, m + 1, r, i, val);
        maintain(node);
    }  // 更新i处的值为val
```

```
    T query(int node, int l, int r, int ql, int qr) const {
        if (ql <= l && r <= qr) return tree[node];
        int m = (l + r) / 2;
        if (qr <= m) return query(node * 2, l, m, ql, qr);
        if (ql > m) return query(node * 2 + 1, m + 1, r, ql, qr);
        T l_res = query(node * 2, l, m, ql, qr);
        T r_res = query(node * 2 + 1, m + 1, r, ql, qr);
        return merge_val(l_res, r_res);
    }  // 查询[ql,qr]的值

    int find_first(int node, int l, int r, int ql, int qr, T val) const {
        if (r < ql || l > qr) return -1;
        if (tree[node].val < val) return -1;
        if (l == r) return l;
        int m = (l + r) >> 1;
        int res = find_first(node << 1, l, m, ql, qr, val);
        if (res != -1) return res;
        return find_first(node << 1 | 1, m + 1, r, ql, qr, val);
    }
    // 若固定左端点，需要记录前缀分段最大值，并加被待求区间完全覆盖的剪枝

    int find_last(int node, int l, int r, int ql, int qr, T val) const {
        if (r < ql || l > qr) return -1;
        if (tree[node].val < val) return -1;
        if (l == r) return l;
        int m = (l + r) >> 1;
        int res = find_last(node << 1 | 1, m + 1, r, ql, qr, val);
        if (res != -1) return res;
        return find_last(node << 1, l, m, ql, qr, val);
    }

public:
    SegmentTree(int n, T init_val) : SegmentTree(vector<T>(n, init_val)) {}

    // 传入一个数组维护
    SegmentTree(const vector<T>& a)
        : n(a.size()), tree(2 << bit_width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }

    void update(int i, T val) { update(1, 0, n - 1, i, val); }  // 更新i的值为val

    T query(int ql, int qr) const { return query(1, 0, n - 1, ql, qr); }  // 查询[ql,qr]的值

    T get(int i) const { return query(1, 0, n - 1, i, i); }  // 取出i处的值

    // 查询[ql,qr]中第一个满足条件的下标
    int find_first(int ql, int qr, T val) const { return find_first(1, 0, n - 1, ql, qr, val); }

    // 查询[ql,qr]中最后一个满足条件的下标
    int find_last(int ql, int qr, T val) const { return find_last(1, 0, n - 1, ql, qr, val); }
};
```

2.3.9 树状数组

```
template <typename T = long long>
class Tree {
```

```
    vector<T> tree;

public:
    // 构造函数：初始化大小为 n 的树状数组，初始所有元素值为 val（外部表现为 0-based）
    Tree(int n, T val = 0) : tree(n + 1) {
        for (int i = 1; i <= n; i++) {
            tree[i] += val;
            int nxt = i + (i & -i);
            if (nxt <= n) {
                tree[nxt] += tree[i];
            }
        }
    }

    // 构造函数：使用给定的 vector 在 O(N) 时间内快速初始化建树
    Tree(const vector<T>& data) {
        int n = data.size();
        tree.resize(n + 1);
        for (int i = 1; i <= n; i++) {
            tree[i] += data[i - 1];  // data是 0-based
            int nxt = i + (i & -i);
            if (nxt <= n) {
                tree[nxt] += tree[i];
            }
        }
    }

    // 单点修改：将 0-based 下标 i 处的元素增加 val
    void add(int i, T val = 1) {
        for (++i; i < tree.size(); i += i & (-i)) {
            tree[i] += val;
        }
    }

    // 前缀求和：计算 0-based 下标区间 [0, i] 内的所有元素之和
    T pre(int i) const {
        T res = 0;
        for (++i; i > 0; i &= i - 1) {
            res += tree[i];
        }
        return res;
    }

    // 区间求和：计算 0-based 下标区间 [l, r] 内的所有元素之和
    T query(int l, int r) const {
        if (r < l) {
            return 0;
        }
        return pre(r) - pre(l - 1);  // 当 l=0 时，pre(-1) 会合理地返回 0
    }

    // 树上二分查找：返回满足前缀和 ≥ val 的最小 0-based 下标
    int lower_bound(T val) const {
        int w = bit_width(tree.size() - 1);
        int res = 0;
        T s = 0;
        for (int i = w - 1; i >= 0; i--) {
            int nxt = res + (1 << i);
            if (nxt < tree.size() && tree[nxt] + s < val) {
                res += (1 << i);
```

```
                s += tree[nxt];
            }
        }
        return res;  // 返回 0-based 下标：内部 1-based 下标为 res + 1，因此 0-based 为 res
    }
};
```

2.3.10 字典树 Trie

```
struct Node {
    Node* son[26]{};
    int pre = 0;
    int endcnt = 0;
};
class Trie {
    Node* root = new Node();

    int find(string word) {
        Node* cur = root;
        for (char c : word) {
            c -= 'a';
            if (!cur->son[c]) return 0;
            cur = cur->son[c];
        }
        return cur->endcnt > 0 ? 2 : 1;
    }
    string findpre(string word) {
        Node* cur = root;
        for (int i = 0; i < word.size(); i++) {
            int c = word[i] - 'a';
            if (!cur->son[c]) return "";
            cur = cur->son[c];
            if (cur->endcnt > 0) return word.substr(0, i + 1);
        }
        return "";
    }
    int findequal(string word) {
        Node* cur = root;
        for (char c : word) {
            c -= 'a';
            if (!cur->son[c]) return 0;
            cur = cur->son[c];
        }
        return cur->endcnt;
    }
    int findprefix(string prefix) {
        Node* cur = root;
        for (char c : prefix) {
            c -= 'a';
            if (!cur->son[c]) return 0;
            cur = cur->son[c];
        }
        return cur->pre;
    }
    void destory(Node* node) {
        if (!node) return;
        for (Node* son : node->son) destory(son);
```

```
        delete node;
    }

public:
    // 析构函数
    ~Trie() { destory(root); }

    // 往trie中插入word
    void insert(string word) {
        Node* cur = root;
        cur->pre++;
        for (char c : word) {
            c -= 'a';
            if (!cur->son[c]) cur->son[c] = new Node();
            cur = cur->son[c];
            cur->pre++;
        }
        cur->endcnt++;
    }

    // 查询trie中是否有word的前缀，若有返回，若无返回空串
    string pre(string word) { return findpre(word); }

    // 查询是否存在与word相等的字符串
    bool search(string word) { return find(word) == 2; }

    // 查询是否存在以prefix为前缀的字符串
    bool startsWith(string prefix) { return find(prefix) != 0; }

    // 查询插入了与word相同的数量
    int cntword(string word) { return findequal(word); }

    // 查询插入了以prefix为前缀的数量
    int cntprefix(string prefix) { return findprefix(prefix); }

    // 往trie中删除word
    void erase(string word) {
        Node* cur = root;
        cur->pre--;
        for (char c : word) {
            c -= 'a';
            cur = cur->son[c];
            cur->pre--;
        }
        cur->endcnt--;
    }
};
```

2.3.11 并查集

```
class UnionFind {
    vector<int> fa;
    vector<int> siz;  // 集合大小
public:
    int cc;  // 连通块个数
    UnionFind(int n) : fa(n), siz(n, 1), cc(n) { ranges::iota(fa, 0); }
    int get(int x) {
```

```
        if (fa[x] != x) fa[x] = get(fa[x]);
        return fa[x];
    }
    bool is_same(int x, int y) { return get(x) == get(y); }
    bool merge(int from, int to) {
        int x = get(from), y = get(to);
        if (x == y) return false;
        fa[x] = y;
        siz[y] += siz[x];
        cc--;
        return true;
    }
    int get_size(int x) {  // 查询x所在集合大小
        return siz[get(x)];
    }
};
```

2.3.12 带权并查集

```
template <typename T>
class UnionFind {
public:
    vector<int> fa;
    vector<T> dis;  // 表示x到x所在集合的代表元的距离

    UnionFind(int n) : fa(n), dis(n) {
        for (int i = 0; i <= n - 1; i++) fa[i] = i;
    }

    int get(int x) {
        if (fa[x] != x) {
            int root = get(fa[x]);
            dis[x] += dis[fa[x]];  // 递归更新x到其代表元的距离
            fa[x] = root;
        }
        return fa[x];
    }

    bool same(int x, int y) { return get(x) == get(y); }

    // 计算从from到to的相对距离，需要它们在同一个集合中
    T get_relative_distance(int from, int to) {
        get(from), get(to);
        return dis[from] - dis[to];
    }

    // 合并from和to，新增信息to-from=value
    // 若to和from不在一个集合，则返回true，否则返回是否与当前信息矛盾
    bool merge(int from, int to, T value) {
        int x = get(from), y = get(to);
        if (x == y) return dis[from] - dis[to] == value;
        dis[x] = value + dis[to] - dis[from]; // 更新代表元之间的距离
        fa[x] = y;
        return true;
    }
};
```

2.3.13 FHQ 平衡树

```
#include <bits/stdc++.h>
#define N 400010
#define P 998244353
#define LS tr[w].ls
#define RS tr[w].rs
#define ll long long
#define GC getchar()
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩾ '0' && c ⩽ '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int ro, x, y, tt, n, m;
ll V, K, X, Y, S1, S2;
struct A {
    int ls, rs;
    ll s, z, ra, su, zu, bj;
} tr[N], o;
// s表个数，z表权值
void sc(int w) {
    tr[w].su = (tr[LS].su + tr[RS].su + tr[w].s) % P;
    tr[w].zu = (tr[LS].zu + tr[RS].zu + tr[w].z * tr[w].s) % P;
}
void ad(int w, ll z) {
    tr[w].z += z;
    tr[w].bj += z;
    tr[w].zu = (tr[w].zu + z * tr[w].su) % P;
    tr[w].zu = (tr[w].zu + P) % P;
}
void xc(int w) {
    ad(LS, tr[w].bj);
    ad(RS, tr[w].bj);
    tr[w].bj = 0;
}
void sp(int w, ll z, int &x, int &y) {
    if (!w) {
        x = y = 0;
        return;
    }
    xc(w);
    if (tr[w].z ⩽ z)
        x = w, sp(RS, z, RS, y);
    else
```

```
        y = w, sp(LS, z, x, LS);
    sc(w);
}
int me(int x, int y) {
    if (!x || !y) return x | y;
    xc(x), xc(y);
    if (tr[x].ra > tr[y].ra) {
        tr[x].rs = me(tr[x].rs, y);
        sc(x);
        return x;
    } else {
        tr[y].ls = me(x, tr[y].ls);
        sc(y);
        return y;
    }
}
int bu(ll S, ll Z) {
    tr[++tt].ra = rand() * rand() + rand();
    tr[tt].s = tr[tt].su = S;
    tr[tt].z = Z;
    tr[tt].zu = S * Z % P;
    return tt;
}
void ins(ll Z) {
    sp(ro, Z, x, y);
    ro = me(me(x, bu(1, Z)), y);
}
int main() {
    srand(time(0));
    n = re(), m = re();
    K = re();
    for (int i = 1; i <= n; i++) {
        V = re();
        ins(V);
    }
    for (int i = 1; i <= m; i++) {
        X = re(), Y = re();
        if (X > 0) {
            sp(ro, K - X, x, y);
            ad(x, X);
            S1 = tr[y].su;
            S2 = tr[y].zu + tr[y].su * X - tr[y].su * K;
            S2 = ((S2 % P) + P) % P;
            // cout<<S1<<' '<<S2<<endl;
            ro = me(me(bu(S2, 1), x), bu(S1, K));
        }
        if (X < 0) {
            sp(ro, -X, x, y);
            ad(y, X);
            ro = y;
        }
        if (Y) ins(Y);
        // cout<<tr[ro].su<<' '<<tr[ro].zu<<endl;
        ks(tr[ro].zu), putchar('\n');
    }
}
```

2.3.14 笛卡尔树（队友）

```cpp
#include <bits/stdc++.h>
#define ll long long
#define N 10000100
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = getchar();
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = getchar();
    }
    while (c <= '9' && c >= '0') s = (s << 1) + (s << 3) + (c ^ 48), c = getchar();
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) + '0');
}
int n, zh[N], zd, bj, ls[N], rs[N], a[N];
ll ans1, ans2;
int main() {
    n = re();
    for (int i = 1; i <= n; i++) {
        a[i] = re();
        bj = 0;
        while (zd && a[zh[zd]] > a[i]) {
            if (bj) rs[zh[zd]] = bj;
            bj = zh[zd--];
        }
        if (bj) ls[i] = bj;
        zh[++zd] = i;
    }
    while (zd > 1) {
        rs[zh[zd - 1]] = zh[zd];
        zd--;
    }
    for (ll i = 1; i <= n; i++) {
        ans1 ^= (ll)i * (ls[i] + 1);
        ans2 ^= (ll)i * (rs[i] + 1);
    }
    cout << ans1 << ' ' << ans2;
}
```

2.3.15 线段树分裂

```cpp
#include <bits/stdc++.h>
#define ll long long
#define N 400010
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
```

```
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, m, ro[N], zh[N << 5], zd, nn, so[N << 5][2], op, x, y, js = 1;
ll z, va[N << 5];
int bu() {
    if (zd) {
        return zh[zd--];
    }
    return ++nn;
}
void xg(int &p, int L, int R, int X, ll Z) {
    if (!p) p = bu();
    va[p] += Z;
    if (L == R) return;
    int mi = (L + R) >> 1;
    if (X <= mi)
        xg(so[p][0], L, mi, X, Z);
    else
        xg(so[p][1], mi + 1, R, X, Z);
}
ll cx(int p, int L, int R, int X) {
    if (X < 1) return 0;
    if (R <= X) return va[p];
    int mi = (L + R) >> 1;
    ll s = 0;
    if (so[p][0]) s += cx(so[p][0], L, mi, X);
    if (so[p][1] && mi < X) s += cx(so[p][1], mi + 1, R, X);
    return s;
}
void sp(int x, int &y, ll z) {
    if (!x) return;
    y = bu();
    ll v = va[so[x][0]];
    if (z > v)
        sp(so[x][1], so[y][1], z - v);
    else
        swap(so[x][1], so[y][1]);
    if (z < v) sp(so[x][0], so[y][0], z);
    va[y] = va[x] - z;
    va[x] = z;
}
int kth(int p, int L, int R, ll z) {
    if (L == R) return L;
    int mi = (L + R) >> 1;
    if (va[so[p][0]] >= z)
        return kth(so[p][0], L, mi, z);
    else
        return kth(so[p][1], mi + 1, R, z - va[so[p][0]]);
```

```
}
void del(int x) {
    zh[++zd] = x;
    so[x][0] = so[x][1] = va[x] = 0;
}
int me(int x, int y) {
    if (!x || !y) return x | y;
    va[x] += va[y];
    so[x][0] = me(so[x][0], so[y][0]);
    so[x][1] = me(so[x][1], so[y][1]);
    del(y);
    return x;
}
int main() {
    n = re(), m = re();
    for (int i = 1; i <= n; i++) {
        z = re();
        xg(ro[1], 1, n, i, z);
    }
    for (int i = 1; i <= m; i++) {
        op = re();
        if (op == 0) {
            z = re();
            x = re(), y = re();
            ll s1 = cx(ro[z], 1, n, x - 1), s2 = cx(ro[z], 1, n, y);
            // cout<<s1<<' '<<s2-s1<<endl;
            s2 -= s1;
            int tm = 0;
            sp(ro[z], ro[++js], s1);
            sp(ro[js], tm, s2);
            ro[z] = me(ro[z], tm);
        }
        if (op == 1) {
            x = re(), y = re();
            ro[x] = me(ro[x], ro[y]);
        }
        if (op == 2) {
            x = re(), z = re(), y = re();
            xg(ro[x], 1, n, y, z);
        }
        if (op == 3) {
            z = re();
            x = re(), y = re();
            ks(cx(ro[z], 1, n, y) - cx(ro[z], 1, n, x - 1));
            PC('\n');
        }
        if (op == 4) {
            x = re(), z = re();
            if (va[ro[x]] < z)
                ks(-1);
            else
                ks(kth(ro[x], 1, n, z));
            PC('\n');
        }
    }
}
```

2.3.16 高维前缀和

```cpp
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 1100010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩾ '0' && c ⩽ '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    putchar((s % 10) | 48);
}
ll dq[1 << 20], su[1 << 20], ss[1 << 20];
int main() {
    for (int i = 1; i < (1 << 5); i++) su[i] = ss[i] = dq[i] = i;
    for (int j = 0; j < 5; j++)
        for (int i = 0; i < (1 << 5); i++)
            if (i & (1 << j)) su[i] = (su[i] + su[i ^ (1 << j)]);
    for (int i = 0; i < (1 << 5); i++) cout << su[i] << ' ';
    cout << endl;
    for (int j = 0; j < 5; j++)
        for (int i = 0; i < (1 << 5); i++)
            if (!(i & (1 << j))) ss[i] = (ss[i] + ss[i ^ (1 << j)]);
    for (int i = 0; i < (1 << 5); i++) cout << ss[i] << ' ';
}
```

2.4 数学

2.4.1 位运算集合

```cpp
// 交集 a&b
// 并集 a|b
// 对称差 a^b
// 相对补 a&(~b)
// 包含 a&b=a or a|b=b

// 集合大小 __builtin_popcount/__builtin_popcountll(s)
// 二进制长度 __lg(s)+1
// 集合最大元素 __lg(s)
// 集合最小元素 __builtin_ctz(s) 需要保证s≠0

// 遍历集合
```

```cpp
for (int x = 0; i < n; x++) {
    if ((s >> x) & 1) {
        // operations
    }
}

// 枚举所有集合
for (int x = 0; x < (1 << n); x++) {
    // operations
}

// 枚举非空子集
for (int x = s; x; x = (x - 1) & s) {
    // operations
}

// 枚举所有子集
int x = s;
while (1) {
    if (x == 0) break;
    x = (x - 1) & s;
}

// 枚举超集
for (int x = s; x < (1 << n); x = (x + 1) | s) {
    // operations
}
```

2.4.2 组合数学

```cpp
constexpr int MOD = 1e9 + 7;
constexpr int MX = 1e5 + 1;
ll F[MX];      // 预处理阶乘
ll INV_F[MX];  // 预处理逆元
ll qpow(ll x, int n) {
    ll res = 1;
    for (; n; n >>= 1) {
        if (n % 2) res = res * x % MOD;
        x = x * x % MOD;
    }
    return res;
}
auto init = [] {
    F[0] = 1;
    for (int i = 1; i < MX; i++) F[i] = F[i - 1] * i % MOD;  // 预处理阶乘
    INV_F[MX - 1] = qpow(F[MX - 1], MOD - 2);
    for (int i = MX - 1; i; i--) {
        INV_F[i - 1] = INV_F[i] * i % MOD;
    }  // 预处理逆元
    return 0;
}();
// 计算C(n,m),即从n个数中取m个数
ll comb(int n, int m) { return m < 0 || m > n ? 0 : F[n] * INV_F[m] % MOD * INV_F[n - m] % MOD; }

constexpr int MX = 31;
int c[MX][MX];  // 即为C(n,m),从n个数中取m个数
auto init = [] {
```

```
    for (int i = 0; i < MX; i++) {
        c[i][0] = c[i][i] = 1;
        for (int j = 1; j < i; j++) {
            c[i][j] = c[i - 1][j - 1] + c[i - 1][j];
        }
    }
    return 0;
}();  // 适用于MX较小的情况
```

2.4.3 约数枚举

```
constexpr int MX = 2e5 + 5;
vector<int> divisors[MX];
auto init = [] {
    for (int i = 1; i < MX; i++) {
        for (int j = i; j < MX; j += i) {
            divisors[j].push_back(i);
        }
    }
    return 0;
}();
```

2.4.4 数论分块

思路：`n / i` 的取值只有 `O(sqrt(n))` 种。
固定左端点 `l` 时，令 `v = n / l`，满足 `n / i = v` 的 `i` 会连续到 `r = n / v`。

用法：

- `floor_sum(n)` 计算 $\sum_{i=1}^{n}\lfloor n/i \rfloor$。
- 如果要计算别的式子，把循环里的 `[l, r]` 和 `v = n / l` 拿去改贡献即可。

```
ll floor_sum(ll n) {
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll v = n / l;
        r = n / v;
        res += v * (r - l + 1);
    }
    return res;
}

void divide_block(ll n) {
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll v = n / l;
        r = n / v;
        // 此时 i in [l, r] 的 n / i 都等于 v
    }
}
```

2.4.5 矩阵快速幂

```
constexpr int MOD = 1e9 + 7;
using matrix = vector<vector<ll>>;
matrix mul(matrix& a, matrix& b) {
    int n = a.size(), m = b[0].size();
    matrix c = matrix(n, vector<ll>(m));
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < a[i].size(); k++) {
            if (a[i][k] == 0) continue;
            for (int j = 0; j < m; j++) {
                c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;
            }
        }
    }
    return c;
}
matrix pow_mul(matrix a, int n, matrix& f0) {
    matrix res = f0;
    while (n) {
        if (n & 1) res = mul(a, res);
        a = mul(a, a);
        n >>= 1;
    }
    return res;
}
```

2.4.6 欧拉函数

```
constexpr int MX = 1e5 + 1;
int phi[MX];
auto init = [] {
    auto Phi = [](int n) -> int {
        int res = n;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                res = res / i * (i - 1);
                while (n % i == 0) {
                    n /= i;
                }
            }
        }
        if (n > 1) res = res / n * (n - 1);
        return res;
    };
    rep(i, 1, MX - 1) { phi[i] = Phi(i); }
    return 0;
}();
```

2.4.7 质因数分解

```cpp
constexpr int MX = 1e7 + 1;
int lpf[MX]; // 存储每个数的最小素因子，复杂度O(NloglogN)
auto init = [] {
    for (int i = 2; i < MX; i++) {
        if (lpf[i] == 0) {
            for (int j = i; j < MX; j += i) {
                if (lpf[j] == 0) lpf[j] = i;
            }
        }
    }
    return 0;
}();
// 质因数分解，返回值为pair<素因子，素因子次幂>，复杂度O(logN)
vector<pair<int, int>> cnt(int x) {
    vector<pair<int, int>> res;
    while (x > 1) {
        int p = lpf[x];
        int e = 1;
        for (x /= p; x % p == 0; x /= p) {
            e++;
        }
        res.emplace_back(p, e);
    }
    return res;
}
```

2.4.8 快速幂

```cpp
constexpr int MOD = 1e9 + 7;
int mul(int x, int y) { return x * 1LL * y % MOD; }
int qpow(int x, int y) {
    int z = 1;
    while (y > 0) {
        if (y & 1) z = mul(z, x);
        x = mul(x, x);
        y >>= 1;
    }
    return z;
}  // 求x**y%MOD

// 注意：当MOD为质数时， (x/y)%MOD=(x*(y**(MOD-2)))%MOD，即y在模MOD意义下的逆元为b^{-1} \equiv b^{p-2} mod p
```

2.4.9 埃氏筛

```cpp
constexpr int M = 2e8;
bool is_prime[M];
vector<int> primes;
auto init = [] {
    ranges::fill(is_prime, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i < M; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
```

```
            for (ll j = 1LL * i * i; j < M; j += i) {
                is_prime[j] = false;
            }
        }
    }
    return 0;
}();
// 函数指针在创建时自动调用
```

2.4.10 欧拉筛

```
constexpr int MX = 6e6 + 1;
bool pd[MX];
vi primes;
auto init = [] {
    rep(i, 2, MX - 1) {
        if (!pd[i]) primes.push_back(i);
        for (int& p : primes) {
            if (1LL * p * i >= MX) break;
            pd[i * p] = true;
            if (i % p == 0) break;
        }
    }
    return 0;
}();
```

2.4.11 线性基（我的）

```
// 模板题，最大子序列异或和
class XorBasis {
    vi b;

public:
    XorBasis(int n) : b(n) {}
    void insert(int x) {
        frep(i, sz(b) - 1, 0) {
            if (x >> i) {
                if (b[i] == 0) {
                    b[i] = x;
                    return;
                }
                x ^= b[i];
            }
        }
    }
    int max_xor() {
        int res = 0;
        frep(i, sz(b) - 1, 0) { res = max(res, res ^ b[i]); }
        return res;
    }
};
class Solution {
public:
    int maxXorSubsequences(vector<int>& nums) {
```

```
        int m = bit_width((uint32_t)ranges::max(nums));
        XorBasis b(m);
        for (int x : nums) b.insert(x);
        return b.max_xor();
    }
};
```

2.4.12 组合计数公式

错排公式：

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

卡特兰数：

$$H_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

第二类斯特林数：

$$f_{i,j} = f_{i-1,j-1} + j f_{i-1,j}, \quad f_{0,0} = 1$$

通项：

$$f_{n,m} = \frac{1}{m!}\sum_{i=0}^{m}(-1)^i\binom{m}{i}(m-i)^n$$

2.4.13 扩展欧几里得

思路：递归求出 $bx' + (a \bmod b)y' = gcd(a,b)$，回代得到 $ax + by = gcd(a,b)$。

用法：

- `ll g = exgcd(a, b, x, y);`
- 返回 `g = gcd(a, b)`，并让 `x, y` 满足 `a * x + b * y = g`。
- 线性同余 `a * x = c (mod m)` 有解当且仅当 `c % g == 0`，一组解为 `x = x * (c / g) mod (m / g)`。

```
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) {
        x = 1;
        y = 0;
```

```
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

ll mod_equation(ll a, ll c, ll m) {
    ll x, y;
    ll g = exgcd(a, m, x, y);
    if (c % g) return -1;  // 无解
    ll mod = m / g;
    x = (__int128)x * (c / g) % mod;
    return (x + mod) % mod;
}
```

2.4.14 卢卡斯定理

思路：当 `p` 是质数时，把 `n, m` 拆成 `p` 进制位：

$$C(n,m) \equiv C(n/p, m/p)C(n \bmod p, m \bmod p) \pmod p$$

用法：

- 先 `init_lucas(p)` 预处理 `0..p-1` 的阶乘和逆阶乘。
- 再调用 `lucas(n, m, p)` 求 `C(n, m) mod p`。
- 注意 `p` 必须是质数，且预处理复杂度是 `O(p)`。

```
vl fac, ifac;

ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void init_lucas(int p) {
    fac.assign(p, 1);
    ifac.assign(p, 1);
    rep(i, 1, p - 1) fac[i] = fac[i - 1] * i % p;
    ifac[p - 1] = qpow(fac[p - 1], p - 2, p);
    frep(i, p - 2, 0) ifac[i] = ifac[i + 1] * (i + 1) % p;
}

ll C_mod_p(ll n, ll m, int p) {
    if (m < 0 || m > n) return 0;
```

```
    return fac[n] * ifac[m] % p * ifac[n - m] % p;
}

ll lucas(ll n, ll m, int p) {
    if (m < 0 || m > n) return 0;
    if (!m) return 1;
    return lucas(n / p, m / p, p) * C_mod_p(n % p, m % p, p) % p;
}
```

2.4.15 中国剩余定理

```
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩾ '0' && c ⩽ '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    putchar((s % 10) | 48);
}
ll n, a, m, na, nm, x, y, xm, xx;
void exgcd(ll a, ll b) {
    if (!b) {
        x = 1;
        y = 0;
        return;
    }
    exgcd(b, a % b);
    ll x1 = x;
    x = y;
    y = x1 - a / b * y;
}
ll gcd(ll a, ll b) {
    if (!b) return a;
    return gcd(b, a % b);
}
int main() {
    n = re(), m = re(), a = re();
    for (int i = 2; i ⩽ n; i++) {
        nm = re(), na = re();
        exgcd(m, nm);
        xm = m / gcd(m, nm) * nm;
        x = (x % xm + xm) % xm;  // x为方程的解
        ll s = x * (m / gcd(m, nm)) % xm;
```

```
        s = s * (na - a) % xm;
        a = (a + s) % xm;
        m = xm;
        a = (a % m + m) % m;
    }
    cout << a;
}
```

2.4.16 线性基（队友）

```
#include <iostream>
#define ll long long
using namespace std;
ll n, a[51], p[61], ans;
void cr(ll x) {
    for (int i = 52; i >= 0; i--) {
        if (x >> i & 1) {
            if (!p[i]) {
                p[i] = x;
                break;
            }
            x ^= p[i];
        }
    }
}
int main() {
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= n; i++) cr(a[i]);
    for (int i = 52; i >= 0; i--)
        if ((ans ^ p[i]) > ans) ans ^= p[i];
    cout << ans;
}
```

2.5 图论

2.5.1 同余最短路

思路：选一个数 `mod` 当模数，把每个余数看成点。
从余数 `r` 加上一个数 `w` 会走到 `(r + w) % mod`，边权是 `w`。
Dijkstra 求出到每个余数的最小可达值 `d[r]`，那么所有 `d[r] + k * mod` 都可达。

用法：

- `count_by_mod(limit, mod, steps)`：统计 `[0, limit]` 中能由 `steps` 里的数凑出的非负整数个数。
- 常见“跳楼机”题：有 `x, y, z` 三种移动，问 `1..h` 有多少层能到。

- 可取 `mod = min({x, y, z})`，`steps` 为另外几个数，调用 `count_by_mod(h - 1, mod, steps)`。

```
ll count_by_mod(ll limit, ll mod, vector<ll> steps) {
    const ll INF = (1LL << 62);
    vector<ll> d(mod, INF);
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    d[0] = 0;
    pq.push({0, 0});
    while (!pq.empty()) {
        auto [dis, x] = pq.top();
        pq.pop();
        if (dis != d[x]) continue;
        for (ll w : steps) {
            ll y = (x + w) % mod;
            if (d[y] > dis + w) {
                d[y] = dis + w;
                pq.push({d[y], y});
            }
        }
    }

    ll ans = 0;
    rep(i, 0, mod - 1) {
        if (d[i] <= limit) ans += (limit - d[i]) / mod + 1;
    }
    return ans;
}

ll solve_floors(ll h, ll x, ll y, ll z) {
    if (x == 1 || y == 1 || z == 1) return h;
    vector<ll> a = {x, y, z};
    sort(all(a));
    return count_by_mod(h - 1, a[0], {a[1], a[2]});
}
```

2.5.2 欧拉路径

```
// 无向图欧拉回路/欧拉路径
vi eulerianPathOnUndirectedGraph(int n, int m, vector<vector<pii>> &ma) {
    // ma的第二维是边的编号
    rep(i, 0, n - 1) {
        sort(all(ma[i]), [&](const pii &a, const pii &b) { return a.first < b.first; });
    }
    int st = 0, cnt = 0;
    frep(i, n - 1, 0) {
        if (!ma[i].empty()) {
            if (sz(ma[i]) % 2) {
                st = i;
                cnt++;
            } else if (cnt == 0) {
                st = i;
            }
        }
    }
```

```
    // 分别处理欧拉回路和欧拉路径的情况
    if (cnt > 2) return {};
    vi path;
    vector<bool> vis(m, false);
    vi head(n);
    auto dfs = [&](this auto &&dfs, int v) -> void {
        for (int &i = head[v]; i < sz(ma[v]);) {
            auto [to, id] = ma[v][i++];
            if (vis[id]) continue;
            vis[id] = true;
            dfs(to);
        }
        path.push_back(v);
        return;
    };
    dfs(st);
    ranges::reverse(path);
    return path;
}

// 有向图欧拉回路/欧拉路径
vi eulerianPathOnUndirectedGraph(int n, int m, vector<vector<pii>> &ma) {
    // ma的第二维是边的编号
    vi in(n);  // 入度
    rep(i, 0, n - 1) {
        for (auto p : ma[i]) in[p.first]++;
    }
    rep(i, 0, n - 1) {
        sort(all(ma[i]), [&](const pii &a, const pii &b) { return a.first < b.first; });
    }
    int st = -1, end = -1;
    rep(i, 0, n - 1) {
        int out = sz(ma[i]);
        if (out == in[i] + 1) {
            if (st >= 0) return {};
            st = i;
        }
        if (out + 1 == in[i]) {
            if (end >= 0) return {};
            end = i;
        }
    }  // 这里是欧拉路径
    if (st < 0) {
        st = 0;
        rep(i, 0, n - 1) {
            if (!ma[i].empty()) {
                st = i;
                break;
            }
        }
    }
    // 分别处理欧拉回路和欧拉路径的情况
    vi path;
    vi head(n);
    auto dfs = [&](this auto &&dfs, int v) -> void {
        for (int &i = head[v]; i < sz(ma[v]);) {
            auto [to, id] = ma[v][i++];
            dfs(to);
        }
        path.push_back(v);
```

```
        return;
    };
    dfs(st);
    ranges::reverse(path);
    return path;
}
```

2.5.3 边双连通分量缩点

```
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 5010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, m, x[N], y[N], he[N], tt, df[N], lo[N], odn, iz[N], zd, ans, nd[N], s, ds[N];
struct A {
    int ne, to;
} b[N << 2];
void ad(int x, int y) {
    // cout<<x<<' '<<y<<endl;
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[++tt].to = x;
    b[tt].ne = he[y];
    he[y] = tt;
}
void dfs(int x, int f) {
    df[x] = lo[x] = ++odn;
    iz[++zd] = x;
    for (int i = he[x]; i; i = b[i].ne) {
        if (i == (f ^ 1)) continue;
        int v = b[i].to;
        if (!df[v]) {
            dfs(v, i);
            lo[x] = min(lo[x], lo[v]);
        } else
            lo[x] = min(lo[x], df[v]);
```

```
    }
    if (lo[x] == df[x]) {
        s++;
        while (iz[zd] != x) {
            nd[iz[zd]] = s;
            zd--;
        }
        nd[x] = s;
        zd--;
    }
}
int main() {
    n = re(), m = re();
    tt = 1;
    for (int i = 1; i <= m; i++) {
        x[i] = re(), y[i] = re();
        ad(x[i], y[i]);
    }
    for (int i = 1; i <= n; i++)
        if (!df[i]) dfs(i, 0);
    for (int i = 1; i <= m; i++)
        if (nd[x[i]] ^ nd[y[i]]) ds[nd[x[i]]]++, ds[nd[y[i]]]++;
    for (int i = 1; i <= s; i++)
        if (ds[i] == 1) ans++;
    cout << ((ans + 1) >> 1) << endl;
    // dfs(1,0); cout<<dp[1][K];
}
```

2.5.4 点双连通分量

```
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 500010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, m, x[N], y[N], he[N], tt, df[N], lo[N], odn;
ll si[N], ans[N];
//,iz[N],zd,nd[N],s,ds[N];
```

```
struct A {
    int ne, to;
} b[N << 1];
void ad(int x, int y) {
    // cout<<x<<' '<<y<<endl;
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[++tt].to = x;
    b[tt].ne = he[y];
    he[y] = tt;
}
void dfs(int x, int f) {
    df[x] = lo[x] = ++odn;
    si[x] = 1;
    ll S = 0;  // iz[++zd]=x;
    for (int i = he[x]; i; i = b[i].ne) {
        if (i == (f ^ 1)) continue;
        int v = b[i].to;
        if (!df[v]) {
            dfs(v, i);
            lo[x] = min(lo[x], lo[v]);
            si[x] += si[v];
            if (lo[v] >= df[x]) {
                ans[x] += S * si[v];
                S += si[v];
            }
        } else
            lo[x] = min(lo[x], df[v]);
    }
    ans[x] += (n - S - 1) * S + n - 1;
}
int main() {
    n = re(), m = re();
    tt = 1;
    for (int i = 1; i <= m; i++) {
        x[i] = re(), y[i] = re();
        ad(x[i], y[i]);
    }
    for (int i = 1; i <= n; i++)
        if (!df[i]) dfs(i, 0);
    for (int i = 1; i <= n; i++) ks(ans[i] * 2), putchar('\n');
    // dfs(1,0); cout<<dp[1][K];
}
```

2.5.5 点双连通分量缩点

```
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 2010
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
```

```
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, m, x, y, cb[N][N], he[N], tt, df[N], lo[N], odn, iz[N], zd, bj[N], co[N], ky, ans, an[N];
struct A {
    int ne, to;
} b[N * N];
vector<int> nt;
void ad(int x, int y) {
    // cout<<x<<' '<<y<<endl;
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
    b[++tt].to = x;
    b[tt].ne = he[y];
    he[y] = tt;
}
void rs(int x) {
    if (!ky) return;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (!bj[v]) continue;
        if (co[v] == co[x]) {
            ky = 0;
            return;
        }
        if (co[v] == -1) co[v] = co[x] ^ 1, rs(v);
    }
}
void dfs(int x, int f) {
    // cout<<x<<' ';
    df[x] = lo[x] = ++odn;
    iz[++zd] = x;
    for (int i = he[x]; i; i = b[i].ne) {
        // cout<<x<<' '<<b[i].to<<endl;
        if (i == (f ^ 1)) continue;
        int v = b[i].to;
        if (!df[v]) {
            dfs(v, i);
            lo[x] = min(lo[x], lo[v]);
            // cout<<x<<'f'<<v<<endl;
            if (lo[v] >= df[x]) {
                nt.clear();
                while (iz[zd] != v) {
                    nt.push_back(iz[zd]);
                    bj[iz[zd]] = 1;
                    zd--;
                }
                nt.push_back(iz[zd]);
```

```
                bj[iz[zd]] = 1;
                zd--;
                bj[x] = 1;
                nt.push_back(x);
                // for(int j=0; j<nt.size(); j++)cout<<nt[j]<<' '; cout<<endl;
                co[nt[0]] = 0;
                ky = 1;
                rs(nt[0]);
                if (!ky && nt.size() >= 3)
                    for (int j = 0; j < nt.size(); j++) an[nt[j]] = 1;
                for (int j = 0; j < nt.size(); j++) bj[nt[j]] = 0;
            }
        } else
            lo[x] = min(lo[x], df[v]);
    }  // cout<<x<<' '<<df[x]<<' '<<lo[x]<<endl;
}
int main() {
    while (1) {
        n = re(), m = re();
        if (!n && !m) break;
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++) cb[i][j] = 0;
        tt = 1;
        odn = ans = 0;
        for (int i = 1; i <= n; i++) df[i] = lo[i] = he[i] = 0, co[i] = -1;
        for (int i = 1; i <= m; i++) {
            x = re(), y = re();
            cb[x][y] = cb[y][x] = 1;
        }
        for (int i = 1; i <= n; i++)
            for (int j = i + 1; j <= n; j++)
                if (!cb[i][j]) ad(i, j);
        for (int i = 1; i <= n; i++)
            if (!df[i]) dfs(i, 0);
        for (int i = 1; i <= n; i++) ans += (an[i] == 0);
        cout << ans << endl;
    }

    // dfs(1,0); cout<<dp[1][K];
}
```

2.5.6 网络流

```
tt = 1;
bool sp() {
    for (int i = 1; i <= ch; i++) di[i] = M;
    di[cy] = 0;
    dl[1] = cy;
    h = t = 1;
    while (h <= t) {
        x = dl[h++];
        for (int i = he[x]; i; i = b[i].ne) {
            int v = b[i].to;
            if (di[v] > di[x] + 1 && b[i].z) {
                di[v] = di[x] + 1;
                dl[++t] = v;
            }
```

```
        }
    }
    return di[ch] < M;
}
ll dfs(int x, ll su) {
    if (!su || x == ch) {
        S += su;
        return su;
    }
    ll s = 0;
    for (int &i = cu[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (di[v] == di[x] + 1) {
            ll ts = dfs(v, min(su - s, b[i].z));
            s += ts;
            b[i].z -= ts;
            b[i ^ 1].z += ts;
            if (s == su) return s;
        }
    }
    return s;
}
while (sp()) {
    for (int i = 1; i <= ch; i++) cu[i] = he[i];
    dfs(cy, L);
}
```

2.5.7 有向图强连通分量缩点

```
#include <bits/stdc++.h>
#define ll long long
#define GC getchar()
#define N 210
#define max(x, xx) x > xx ? x : xx
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) putchar('-'), s = -s;
    if (s >= 10) ks(s / 10);
    putchar((s % 10) | 48);
}
int n, m, x[N], y[N], he[N], tt, df[N], lo[N], odn, zh[N], zd, ans, nd[N], rd[N], s, iz[N];
struct A {
    int ne, to;
} b[N * N];
```

```
void ad(int x, int y) {
    // cout<<x<<' '<<y<<endl;
    b[++tt].to = y;
    b[tt].ne = he[x];
    he[x] = tt;
}
void dfs(int x) {
    df[x] = lo[x] = ++odn;
    zh[++zd] = x;
    iz[x] = 1;
    for (int i = he[x]; i; i = b[i].ne) {
        int v = b[i].to;
        if (!df[v]) {
            dfs(v);
            lo[x] = min(lo[x], lo[v]);
        } else if (iz[v])
            lo[x] = min(lo[x], df[v]);
    }
    if (lo[x] == df[x]) {
        s++;
        while (zh[zd] != x) {
            nd[zh[zd]] = s;
            iz[zh[zd]] = 0;
            zd--;
        }
        nd[x] = s;
        iz[x] = 0;
        zd--;
    }
}
int main() {
    n = re();
    for (int i = 1; i <= n; i++)
        while (1) {
            m = re();
            if (!m) break;
            ad(i, m);
        }
    for (int i = 1; i <= n; i++)
        if (!df[i]) dfs(i);
    for (int i = 1; i <= n; i++)
        for (int j = he[i]; j; j = b[j].ne)
            if (nd[i] != nd[b[j].to]) rd[nd[b[j].to]]++;
    for (int i = 1; i <= s; i++)
        if (!rd[i]) ans++;
    cout << ans << endl;
    // dfs(1,0); cout<<dp[1][K];
}
```

2.6 字符串

2.6.1 字符串哈希

```
using ull = unsigned long long;
```

```cpp
ull splitmix64(ull x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}

struct custom_hash {
    static const ull FIXED_RANDOM;

    size_t operator()(ull x) const {
        return splitmix64(x + FIXED_RANDOM);
    }
};

const ull custom_hash::FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();

struct StringHash {
    static ull base() {
        static ull b = splitmix64(chrono::steady_clock::now().time_since_epoch().count()) | 1;
        return b;
    }

    vector<ull> h, p;

    StringHash(const string& s) {
        int n = s.size();
        h.assign(n + 1, 0);
        p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = h[i] * base() + (unsigned char)s[i] + 1;
            p[i + 1] = p[i] * base();
        }
    }

    // 获取 s[l...r] 的哈希值，闭区间，0-indexed
    ull query(int l, int r) const {
        return h[r + 1] - h[l] * p[r - l + 1];
    }
};

// 使用：StringHash hs(s); hs.query(l, r);
// 使用：unordered_map<ull, int, custom_hash> ma;
```

2.6.2 KMP

```cpp
vector<int> kmp(const string& text, const string& pattern) {
    int m = pattern.size();
    vector<int> pi(m);
    int cnt = 0;
    for (int i = 1; i < m; i++) {
        char b = pattern[i];
        while (cnt && pattern[cnt] != b) cnt = pi[cnt - 1];
        if (pattern[cnt] == b) cnt++;
        pi[i] = cnt;
    }
    vector<int> res;
```

```
    cnt = 0;
    for (int i = 0; i < text.size(); i++) {
        char b = text[i];
        while (cnt && pattern[cnt] != b) {
            cnt = pi[cnt - 1];
        }
        if (pattern[cnt] == b) cnt++;
        if (cnt == m) {
            res.push_back(i - m + 1);
            cnt = pi[cnt - 1];
        }
    }
    return res;
}

auto kmp = [&](const string& pattern) -> vi {
    int m = sz(pattern);
    int cnt = 0;
    vi pi(m);
    rep(i, 1, m - 1) {
        char b = pattern[i];
        while (cnt && pattern[cnt] != b) cnt = pi[cnt - 1];
        if (pattern[cnt] == b) cnt++;
        pi[i] = cnt;
    }
    return pi;
};
```

2.6.3 Z 函数

```
vector<int> zfunc(const string& s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            l = i, r = i + z[i];
            z[i]++;
        }
    }
    z[0] = n;
    return z;
}
```

2.6.4 后缀自动机 SAM

```
#include <bits/stdc++.h>
#define ll long long
#define N 200010
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
```

```
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, z, la = 1, cn = 1, l[N], fa[N];
ll ans;
map<int, int> ne[N];
void ins(int c) {
    int p = la, np = ++cn;
    la = np;
    l[np] = l[p] + 1;
    for (; !ne[p][c]; p = fa[p]) ne[p][c] = np;
    if (!p)
        fa[np] = 1;
    else {
        int q = ne[p][c];
        if (l[q] == l[p] + 1)
            fa[np] = q;
        else {
            int nq = ++cn;
            l[nq] = l[p] + 1;
            ne[nq] = ne[q];
            fa[nq] = fa[q];
            fa[q] = fa[np] = nq;
            for (; ne[p][c] == q; p = fa[p]) ne[p][c] = nq;
        }
    }
    ans += l[np] - l[fa[np]];
}
int main() {
    n = re();
    for (int i = 1; i <= n; i++) {
        z = re();
        ins(z);
        ks(ans), PC('\n');
    }
}
```

2.6.5 回文自动机 PAM

```
#include <bits/stdc++.h>
#define ll long long
#define N 500010
#define M 99999999
```

```
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩽ '9' && c ⩾ '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    PC((s % 10) | 48);
}
char a[N];
int n, fa[N], le[N], ans, cu, ds = 1, po, tr[N][26], gs[N];
int g_f(int x, int w) {
    while (a[w - le[x] - 1] ≠ a[w]) x = fa[x];
    return x;
}
int main() {
    cin >> (a + 1);
    n = strlen(a + 1);
    fa[0] = 1;
    le[1] = -1;
    for (int i = 1; i ⩽ n; i++) {
        if (i ⩾ 2) a[i] = ((a[i] - 'a' + ans) % 26) + 'a';
        po = g_f(cu, i);
        if (!tr[po][a[i] - 'a']) {
            fa[++ds] = tr[g_f(fa[po], i)][a[i] - 'a'];
            tr[po][a[i] - 'a'] = ds;
            le[ds] = le[po] + 2;
            gs[ds] = gs[fa[ds]] + 1;
        }
        cu = tr[po][a[i] - 'a'];
        ans = gs[cu];
        ks(ans), PC(' ');
    }
}
```

2.6.6 广义后缀自动机广义 SAM

```
#include <bits/stdc++.h>
#define ll long long
#define N 2000100
#define P 1000000007
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
```

```
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c >= '0' && c <= '9') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
int n, la, h, O = 1, li[N], ml[N], tr[N][26];
char a[N];
ll ans;
int ins(int c, int la) {
    if (tr[la][c]) {
        int p = la, x = tr[la][c];
        if (ml[p] + 1 == ml[x])
            return x;
        else {
            int y = ++O;
            ml[y] = ml[p] + 1;
            for (int i = 0; i < 26; i++) tr[y][i] = tr[x][i];
            while (p && tr[p][c] == x) tr[p][c] = y, p = li[p];
            li[y] = li[x];
            li[x] = y;
            return y;
        }
    }
    int z = ++O, p = la;
    ml[z] = ml[p] + 1;
    while (p && !tr[p][c]) tr[p][c] = z, p = li[p];
    if (!p)
        li[z] = 1;
    else {
        int x = tr[p][c];
        if (ml[p] + 1 == ml[x])
            li[z] = x;
        else {
            int y = ++O;
            ml[y] = ml[p] + 1;
            for (int i = 0; i < 26; i++) tr[y][i] = tr[x][i];
            while (p && tr[p][c] == x) tr[p][c] = y, p = li[p];
            li[y] = li[x];
            li[z] = li[x] = y;
        }
    }
    return z;
}
int main() {
    n = re();
    for (int i = 1; i <= n; i++) {
        cin >> (a + 1);
        la = 1;
        h = strlen(a + 1);
        for (int j = 1; j <= h; j++) la = ins(a[j] - 'a', la);
    }
    for (int i = 2; i <= O; i++) ans += ml[i] - ml[li[i]];
```

```
    cout << ans;
}
```

2.7 多项式

2.7.1 多项式指数 exp

```
#include <bits/stdc++.h>
#define N 100010
#define P 998244353
#define ll long long
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, L, s;
ll f[N], g[N], R[N], ny_A[N], ln_A[N], ln_B[N], exp_A[N], exp_B[N], ny[N];
ll ksm(ll ds, ll zs) {
    ll s = 1;
    while (zs) {
        if (zs & 1) s = s * ds % P;
        ds = ds * ds % P;
        zs >>= 1;
    }
    return s;
}
void pre(int n) {
    L = 1, s = 0;
    while (L < (n << 1)) {
        L <<= 1;
        s++;
    }
    for (int i = 0; i < L; i++) R[i] = (R[i >> 1] >> 1) | ((i & 1) << (s - 1));
}
void NTT(ll *A, int n, int op) {
    for (int i = 0; i < n; i++)
        if (i < R[i]) swap(A[i], A[R[i]]);
    for (int i = 1; i < n; i <<= 1) {
```

```
        ll W = ksm(3, (P - 1) / (i << 1));
        if (op == -1) W = ksm(W, P - 2);
        for (int j = 0; j < n; j += (i << 1)) {
            for (ll k = 0, w = 1; k < i; k++, w = w * W % P) {
                ll X = A[j + k], Y = A[j + k + i] * w % P;
                A[j + k] = (X + Y) % P;
                A[j + k + i] = (X - Y + P) % P;
            }
        }
    }
    if (op == -1) {
        ll z = ksm(n, P - 2);
        for (int i = 0; i < n; i++) A[i] = A[i] * z % P;
    }
}
void g_n(ll *A, ll *B, int n) {
    if (n == 1) {
        B[0] = ksm(A[0], P - 2);
        return;
    }
    g_n(A, B, n >> 1);
    pre(n);
    for (int i = n; i < L; i++) B[i] = 0;
    for (int i = 0; i < n; i++) ny_A[i] = A[i];
    for (int i = n; i < L; i++) ny_A[i] = 0;
    NTT(B, L, 1);
    NTT(ny_A, L, 1);
    for (int i = 0; i < L; i++) B[i] = (P + 2 - ny_A[i] * B[i] % P) * B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
void g_d(ll *A, ll *B, int n) {
    for (int i = 0; i < n - 1; i++) B[i] = 1ll * A[i + 1] * (i + 1) % P;
}
void jf(ll *A, int n) {
    for (int i = n - 1; i >= 1; i--) A[i] = 1ll * A[i - 1] * ny[i] % P;
    A[0] = 0;
}
void g_ln(ll *A, ll *B, int n) {
    g_n(A, ln_A, n);
    g_d(A, ln_B, n);
    pre(n);
    for (int i = n; i < L; i++) ln_A[i] = ln_B[i] = 0;
    NTT(ln_A, L, 1);
    NTT(ln_B, L, 1);
    for (int i = 0; i < L; i++) B[i] = ln_A[i] * ln_B[i] % P;
    NTT(B, L, -1);
    jf(B, L);
    for (int i = n; i < L; i++) B[i] = 0;
}
void g_exp(ll *A, ll *B, int n) {
    if (n == 1) {
        B[0] = 1;
        return;
    }
    g_exp(A, exp_A, n >> 1);
    g_ln(exp_A, exp_B, n);
    pre(n);
    for (int i = n; i < L; i++) exp_A[i] = exp_B[i] = 0;
    for (int i = 0; i < n; i++) exp_B[i] = (P - exp_B[i] + A[i]) % P;
```

```
    exp_B[0] = (exp_B[0] + 1) % P;
    NTT(exp_A, L, 1);
    NTT(exp_B, L, 1);
    for (int i = 0; i < L; i++) B[i] = exp_A[i] * exp_B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
int main() {
    ny[1] = 1;
    for (int i = 2; i < N; i++) ny[i] = (P - P / i) * ny[P % i] % P;
    n = re();
    int cd = 1;
    for (; cd < (n << 1); cd <<= 1);
    for (int i = 0; i < n; i++) f[i] = re();
    g_exp(f, g, cd);
    for (int i = 0; i < n; i++) ks(g[i]), PC(' ');
}  // 5 1 6 3 4 9
// 1 998244347 33 998244169 1020
```

2.7.2 多项式对数 ln

```
#include <bits/stdc++.h>
#define N 100010
#define P 998244353
#define ll long long
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, L, s;
ll f[N], g[N], R[N], ny_A[N], ln_A[N], ln_B[N];
ll ksm(ll ds, ll zs) {
    ll s = 1;
    while (zs) {
        if (zs & 1) s = s * ds % P;
        ds = ds * ds % P;
        zs >>= 1;
    }
    return s;
}
```

```
void pre(int n) {
    L = 1;
    s = 0;
    while (L < (n << 1)) {
        L <<= 1;
        s++;
    }
    for (int i = 0; i < L; i++) R[i] = (R[i >> 1] >> 1) | ((i & 1) << (s - 1));
}
void NTT(ll *A, int n, int op) {
    for (int i = 0; i < n; i++)
        if (i < R[i]) swap(A[i], A[R[i]]);
    for (int i = 1; i < n; i <<= 1) {
        ll W = ksm(3, (P - 1) / (i << 1));
        if (op == -1) W = ksm(W, P - 2);
        for (int j = 0; j < n; j += (i << 1)) {
            for (ll k = 0, w = 1; k < i; k++, w = w * W % P) {
                ll X = A[j + k], Y = A[j + k + i] * w % P;
                A[j + k] = (X + Y) % P;
                A[j + k + i] = (X - Y + P) % P;
            }
        }
    }
    if (op == -1) {
        ll z = ksm(n, P - 2);
        for (int i = 0; i < n; i++) A[i] = A[i] * z % P;
    }
}
void g_n(ll *A, ll *B, int n) {
    if (n == 1) {
        B[0] = ksm(A[0], P - 2);
        return;
    }
    g_n(A, B, n >> 1);
    pre(n);
    for (int i = n; i < L; i++) B[i] = 0;
    for (int i = 0; i < n; i++) ny_A[i] = A[i];
    for (int i = n; i < L; i++) ny_A[i] = 0;
    NTT(B, L, 1);
    NTT(ny_A, L, 1);
    for (int i = 0; i < L; i++) B[i] = (P + 2 - ny_A[i] * B[i] % P) * B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
void g_d(ll *A, ll *B, int n) {
    for (int i = 0; i < n - 1; i++) B[i] = 1ll * A[i + 1] * (i + 1) % P;
    B[n - 1] = 0;
}
void jf(ll *A, int n) {
    for (int i = n - 1; i >= 1; i--) A[i] = 1ll * A[i - 1] * ksm(i, P - 2) % P;
    A[0] = 0;
}
void g_ln(ll *A, ll *B, int n) {
    g_n(A, ln_A, n);
    g_d(A, ln_B, n);
    pre(n);
    for (int i = n; i < L; i++) ln_A[i] = ln_B[i] = 0;
    // for(int i=0; i<n; i++)cout<<ln_A[i]<<' '; cout<<endl;
    // for(int i=0; i<n; i++)cout<<ln_B[i]<<' '; cout<<endl;
    NTT(ln_A, L, 1);
```

```
    NTT(ln_B, L, 1);
    for (int i = 0; i < L; i++) B[i] = ln_A[i] * ln_B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
    jf(B, n);
}
int main() {
    n = re();
    int cd = 1;
    for (; cd < (n << 1); cd <<= 1);
    for (int i = 0; i < n; i++) f[i] = re();
    g_ln(f, g, cd);
    for (int i = 0; i < n; i++) ks(g[i]), PC(' ');
}  // 5 1 6 3 4 9
// 1 998244347 33 998244169 1020
```

2.7.3 多项式开根 sqrt

```
#include <bits/stdc++.h>
#define N 100010
#define P 998244353
#define ll long long
#define GC getchar()
#define PC putchar
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c <= '9' && c >= '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s >= 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, L, s;
ll f[N], g[N], R[N], ny_A[N], ln_A[N], ln_B[N], exp_A[N], exp_B[N], ny[N], sq_A[N], sq_B[N], sq_C[N];
ll ksm(ll ds, ll zs) {
    ll s = 1;
    while (zs) {
        if (zs & 1) s = s * ds % P;
        ds = ds * ds % P;
        zs >>= 1;
    }
    return s;
}
void pre(int n) {
    L = 1, s = 0;
    while (L < (n << 1)) {
```

```
        L <<= 1;
        s++;
    }
    for (int i = 0; i < L; i++) R[i] = (R[i >> 1] >> 1) | ((i & 1) << (s - 1));
}
void NTT(ll *A, int n, int op) {
    for (int i = 0; i < n; i++)
        if (i < R[i]) swap(A[i], A[R[i]]);
    for (int i = 1; i < n; i <<= 1) {
        ll W = ksm(3, (P - 1) / (i << 1));
        if (op == -1) W = ksm(W, P - 2);
        for (int j = 0; j < n; j += (i << 1)) {
            for (ll k = 0, w = 1; k < i; k++, w = w * W % P) {
                ll X = A[j + k], Y = A[j + k + i] * w % P;
                A[j + k] = (X + Y) % P;
                A[j + k + i] = (X - Y + P) % P;
            }
        }
    }
    if (op == -1) {
        ll z = ksm(n, P - 2);
        for (int i = 0; i < n; i++) A[i] = A[i] * z % P;
    }
}
void g_n(ll *A, ll *B, int n) {
    if (n == 1) {
        B[0] = ksm(A[0], P - 2);
        return;
    }
    g_n(A, B, n >> 1);
    pre(n);
    for (int i = n; i < L; i++) B[i] = 0;
    for (int i = 0; i < n; i++) ny_A[i] = A[i];
    for (int i = n; i < L; i++) ny_A[i] = 0;
    NTT(B, L, 1);
    NTT(ny_A, L, 1);
    for (int i = 0; i < L; i++) B[i] = (P + 2 - ny_A[i] * B[i] % P) * B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
void g_d(ll *A, ll *B, int n) {
    for (int i = 0; i < n - 1; i++) B[i] = 1ll * A[i + 1] * (i + 1) % P;
}
void jf(ll *A, int n) {
    for (int i = n - 1; i >= 1; i--) A[i] = 1ll * A[i - 1] * ny[i] % P;
    A[0] = 0;
}
void g_ln(ll *A, ll *B, int n) {
    g_n(A, ln_A, n);
    g_d(A, ln_B, n);
    pre(n);
    for (int i = n; i < L; i++) ln_A[i] = ln_B[i] = 0;
    NTT(ln_A, L, 1);
    NTT(ln_B, L, 1);
    for (int i = 0; i < L; i++) B[i] = ln_A[i] * ln_B[i] % P;
    NTT(B, L, -1);
    jf(B, L);
    for (int i = n; i < L; i++) B[i] = 0;
}
void g_exp(ll *A, ll *B, int n) {
```

```
    if (n == 1) {
        B[0] = 1;
        return;
    }
    g_exp(A, exp_A, n >> 1);
    pre(n);
    for (int i = 0; i < L; i++) exp_B[i] = 0;
    g_ln(exp_A, exp_B, n);
    pre(n);
    for (int i = n; i < L; i++) exp_A[i] = exp_B[i] = 0;
    for (int i = 0; i < n; i++) exp_B[i] = (P - exp_B[i] + A[i]) % P;
    exp_B[0] = (exp_B[0] + 1) % P;
    NTT(exp_A, L, 1);
    NTT(exp_B, L, 1);
    for (int i = 0; i < L; i++) B[i] = exp_A[i] * exp_B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
ll sq(ll z) { return sqrt(z); }
void g_sq(ll *A, ll *B, int n) {
    if (n == 1) {
        B[0] = sq(A[0]);
        return;
    }
    g_sq(A, sq_A, n >> 1);
    pre(n);
    for (int i = 0; i < L; i++) sq_B[i] = 0;
    g_n(sq_A, sq_B, n);
    for (int i = 0; i < n; i++) sq_C[i] = A[i];
    for (int i = n; i < L; i++) sq_A[i] = sq_B[i] = sq_C[i] = 0;
    NTT(sq_A, L, 1);
    NTT(sq_B, L, 1);
    NTT(sq_C, L, 1);
    for (int i = 0; i < L; i++) B[i] = (sq_C[i] * sq_B[i] + sq_A[i]) % P * (P / 2 + 1) % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
int main() {
    ny[1] = 1;
    for (int i = 2; i < N; i++) ny[i] = (P - P / i) * ny[P % i] % P;
    n = re();
    int cd = 1;
    for (; cd < (n << 1); cd <<= 1);
    for (int i = 0; i < n; i++) f[i] = re();
    g_exp(f, g, cd);
    for (int i = 0; i < n; i++) ks(g[i]), PC(' ');
}  // 5 1 6 3 4 9
// 1 998244347 33 998244169 1020
```

2.7.4 多项式求逆

```
#include <bits/stdc++.h>
#define N 100010
#define P 998244353
#define ll long long
#define GC getchar()
#define PC putchar
```

```
using namespace std;
ll re() {
    ll s = 0, b = 1;
    char c = GC;
    while (c > '9' || c < '0') {
        if (c == '-') b = -b;
        c = GC;
    }
    while (c ⩽ '9' && c ⩾ '0') {
        s = (s << 1) + (s << 3) + (c ^ 48);
        c = GC;
    }
    return s * b;
}
void ks(ll s) {
    if (s < 0) PC('-'), s = -s;
    if (s ⩾ 10) ks(s / 10);
    PC((s % 10) | 48);
}
int n, L, s;
ll f[N], g[N], R[N], ny_A[N];
ll ksm(ll ds, ll zs) {
    ll s = 1;
    while (zs) {
        if (zs & 1) s = s * ds % P;
        ds = ds * ds % P;
        zs ⩾>= 1;
    }
    return s;
}
void pre(int n) {
    L = 1;
    s = 0;
    while (L < (n << 1)) {
        L ⩽<= 1;
        s++;
    }
    for (int i = 0; i < L; i++) R[i] = (R[i >> 1] >> 1) | ((i & 1) << (s - 1));
}
void NTT(ll *A, int n, int op) {
    for (int i = 0; i < n; i++)
        if (i < R[i]) swap(A[i], A[R[i]]);
    for (int i = 1; i < n; i ⩽<= 1) {
        ll W = ksm(3, (P - 1) / (i << 1));
        if (op == -1) W = ksm(W, P - 2);
        for (int j = 0; j < n; j += (i << 1)) {
            for (ll k = 0, w = 1; k < i; k++, w = w * W % P) {
                ll X = A[j + k], Y = A[j + k + i] * w % P;
                A[j + k] = (X + Y) % P;
                A[j + k + i] = (X - Y + P) % P;
            }
        }
    }
    if (op == -1) {
        ll z = ksm(n, P - 2);
        for (int i = 0; i < n; i++) A[i] = A[i] * z % P;
    }
}
void g_n(ll *A, ll *B, int n) {
    if (n == 1) {
```

```
        g[0] = ksm(f[0], P - 2);
        return;
    }
    g_n(A, B, n >> 1);
    pre(n);
    for (int i = n; i < L; i++) B[i] = 0;
    for (int i = 0; i < n; i++) ny_A[i] = A[i];
    for (int i = n; i < L; i++) ny_A[i] = 0;
    NTT(B, L, 1);
    NTT(ny_A, L, 1);
    for (int i = 0; i < L; i++) B[i] = (P + 2 - ny_A[i] * B[i] % P) * B[i] % P;
    NTT(B, L, -1);
    for (int i = n; i < L; i++) B[i] = 0;
}
int main() {
    n = re();
    int cd = 1;
    for (; cd < (n << 1); cd <<= 1);
    for (int i = 0; i < n; i++) f[i] = re();
    g_n(f, g, cd);
    for (int i = 0; i < n; i++) ks(g[i]), PC(' ');
}  // 5 1 6 3 4 9
// 1 998244347 33 998244169 1020
```

2.8 杂项

2.8.1 __int128 输入输出

```
using i128 = __int128;

// 重载输入运算符以支持__int128类型
std::istream &operator>>(std::istream &is, __int128 &val) {
    std::string s;
    bool flag = true;
    is >> s, val = 0;
    if (s[0] == '-') flag = false, val = std::stoi(s.substr(0, 2)), s = s.substr(2);
    for (char &c: s) val = val * 10 + (c - '0') * (!flag ? -1 : 1);
    return is;
}

//重载输出运算符以支持__int128类型
std::ostream &operator<<(std::ostream &os, __int128 val) {
    if (val < 0) os << "-", val = -val;
    if (val > 9) os << val / 10;
    os << static_cast<char>(val % 10 + '0');
    return os;
}
```

2.8.2 对拍器

`gen.cpp` 生成数据，`sol.cpp` 是待测程序，`brute.cpp` 是暴力/标程。
把 `gen.cpp`、`sol.cpp`、`brute.cpp` 和这个对拍器放在同一个目录，编译运行对拍器即可。
如果发现答案不同，它会停下来并保留 `input.txt`、`output.txt`、`answer.txt`，直接看这三个文件定位问题。
`gen.cpp` 要按题目的输入格式随机造一组合法小数据；数据范围故意开小一点，保证 `brute.cpp` 跑得动。

`gen.cpp` 示例，数组题可以先这样写，再按题目改输出格式：

```cpp
#include <bits/stdc++.h>
using namespace std;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

int rnd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

int main() {
    // 数组题示例：按题目输入格式改这里。
    int n = rnd(1, 8);  // 数据要小到 brute.cpp 能跑。
    cout << n << '\n';
    for (int i = 0; i < n; i++) {
        cout << rnd(-10, 10) << " \n"[i == n - 1];
    }
    return 0;
}
```

```cpp
#include <bits/stdc++.h>
using namespace std;

const int TESTS = 1000;

#ifdef _WIN32
const string COMPILE =
    "g++ gen.cpp -std=c++17 -O2 -o gen.exe && "
    "g++ sol.cpp -std=c++17 -O2 -o sol.exe && "
    "g++ brute.cpp -std=c++17 -O2 -o brute.exe";
const string GEN = "gen.exe > input.txt";
const string SOL = "sol.exe < input.txt > output.txt";
const string BRUTE = "brute.exe < input.txt > answer.txt";
const string CHECK = "fc output.txt answer.txt > nul";
#else
const string COMPILE =
    "g++ gen.cpp -std=c++17 -O2 -o gen && "
    "g++ sol.cpp -std=c++17 -O2 -o sol && "
    "g++ brute.cpp -std=c++17 -O2 -o brute";
const string GEN = "./gen > input.txt";
const string SOL = "./sol < input.txt > output.txt";
const string BRUTE = "./brute < input.txt > answer.txt";
const string CHECK = "diff -w output.txt answer.txt > /dev/null";
#endif
```

```cpp
int run(const string& cmd) {
    return system(cmd.c_str());
}

int main() {
    if (run(COMPILE) != 0) {
        cerr << "compile error\n";
        return 1;
    }

    for (int tc = 1; tc <= TESTS; tc++) {
        if (run(GEN) != 0) {
            cerr << "generator failed on test " << tc << '\n';
            return 1;
        }
        if (run(SOL) != 0) {
            cerr << "solution runtime error on test " << tc << '\n';
            return 1;
        }
        if (run(BRUTE) != 0) {
            cerr << "brute runtime error on test " << tc << '\n';
            return 1;
        }
        if (run(CHECK) != 0) {
            cerr << "wrong answer on test " << tc << '\n';
            cerr << "input: input.txt\n";
            cerr << "output: output.txt\n";
            cerr << "answer: answer.txt\n";
            return 1;
        }
        cerr << "accepted on test " << tc << '\n';
    }

    cerr << "all tests passed\n";
    return 0;
}
```

3 Python 常用模板

校赛 OJ 不能用的库保持注释状态，别开 `sortedcontainers` 和 `functools.cache`。

```python
from collections import defaultdict, deque, Counter

# from functools import cache
# from sortedcontainers import SortedSet, SortedDict, SortedList
from typing import List, Tuple, Set
from math import inf
from heapq import heappop, heappush
from bisect import bisect_left, bisect_right
import math
import sys

input = lambda: sys.stdin.readline().rstrip()
```

```python
ii = lambda: int(input())
mii = lambda: map(int, input().split())
lii = lambda: list(mii())


def solve():
    a, b, c = mii()
    print((a + b) * c)


solve()
```

常见用法：

```python
d = defaultdict(int)
q = deque([0]); q.append(1); q.popleft()
cnt = Counter(a); cnt[x] += 1

h = []
heappush(h, (dis, x)); dis, x = heappop(h)

i = bisect_left(a, x)   # 第一个 ≥ x
j = bisect_right(a, x)  # 第一个 > x

ans = inf
g = math.gcd(a, b)      # 最大公约数
l = math.lcm(a, b)      # 最小公倍数
r = math.isqrt(n)       # floor(sqrt(n))，整数开根，不会有浮点误差
up = (a + b - 1) // b   # 正整数向上取整，比 math.ceil(a / b) 稳
down = a // b           # 向下取整
c = math.comb(n, k)     # 组合数 C(n, k)
p = math.perm(n, k)     # 排列数 A(n, k)
```